WNDRVR
SUPPORT NETWORK

VXWORKS 7 IPSEC PROGRAMMER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 IPsec Programmer's Guide, SR0650

12 June 2020

1. OVERVIEW

1. Wind River IPsec

Wind River IPsec is a target- and RTOS-portable implementation of the IP security (IPsec) protocols specified by the Internet Engineering Task Force (IETF). It is designed for use with the VxWorks® Network Stack. It supports both IPv4 and IPv6.

Wind River IPsec interoperates with other RFC-compliant implementations of IPsec; for example, those available on Windows XP, Windows Vista, Linux, and BSD variants, and on Cisco equipment. It is also designed to work with PF_KEY v2-compliant implementations of the Internet Key Exchange (IKE) protocols, such as Wind River IKE. Wind River acknowledges both the OpenBSD project and the Linux project for their PF_KEY v2 extensions.

2. Technology Overview

IPsec is a standardized framework for securing IP communications between network peers, specified in RFC 4301: Security Architecture for the Internet Protocol.

IPsec can provide the following security services:

- access control
- data origin authentication
- connectionless data integrity checking
- replay protection
- confidentiality (by encryption)
- limited traffic flow confidentiality

For more information on the technologies underlying the above capabilities, see How IPsec Works on page 4.

3. Product Overview

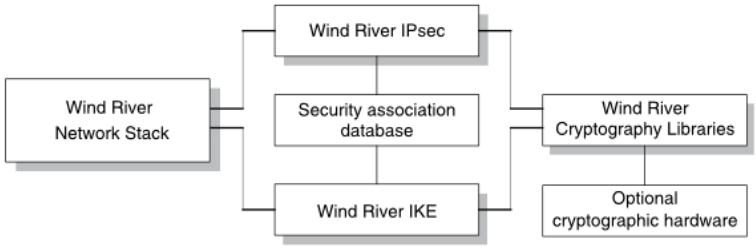
In a network that employs IP data security, the Wind River Network Stack passes IP packets to Wind River IPsec, which verifies cryptographic checksums, and encrypts or decrypts the packets before returning them to the stack.

If required, the instance of IPsec running on a packet's source host negotiates a cryptographic protection suite with the packet's destination host, using an automated process such as that provided by Wind River IKE. The Wind River Cryptography Libraries supply cryptographic algorithm support to VxWorks applications. Wind River products such as Wind River IPsec call the functions as needed. In addition, other applications can call the Cryptography Libraries functions using the standard OpenSSL API.

Wind River IPsec implements the RFCs listed in RFCs on page 2.

The relationships between Wind River IPsec and other Wind River products are expressed in the following diagram.

Figure 1. Product Relationships



4. RFCs

Wind River IPsec complies with the IETF RFCs listed in this section.

The RFCs are accessible from the following Web site:

http://www.ietf.org

IPsec Protocols

- RFC 1826: *IP Authentication Header*
- RFC 1827: *IP Encapsulating Security Payload (ESP)*
- RFC 1828: *IP Authentication using Keyed MD5*
- RFC 1852: *IP Authentication using Keyed SHA*
- RFC 1853: *IP in IP Tunneling*
- RFC 2402: *IP Authentication Header*
- RFC 2406: *IP Encapsulating Security Payload (ESP)*
- RFC 3948: *UDP Encapsulation of IPsec ESP Packets*

> Note: Transport mode and NAT-Traversal must be handled carefully, especially if more than two parties are communicating with each other. RFC 3948 describes some of the issues that arise. When Wind River IKE serves as a client behind NAT and transport mode is used, Wind River recommends that access is restricted to a single client (for example, by specifying the remote_identifier of the client).

- RFC 4301: *Security Architecture for the Internet Protocol*
- RFC 4302: *IP Authentication Header (AH) RFC 4303: IP Encapsulating Security Payload (ESP)*
- RFC 4304: *Extended Sequence Number (ESN) Addendum to IPsec Domain of Interpretation (DOI) for Internet Security Association and Key Management Protocol (ISAKMP)*
- RFC 5858: *IPsec Extensions to Support Robust Header Compression over IPsec*
- **draft-ietf-ipsec-monitor-mib-03.txt** IPsec Monitoring MIB

Cryptographic Algorithms

- RFC 2403: *The Use of HMAC-MD5-96 within ESP and AH*
- RFC 2404: *The Use of HMAC-SHA-1-96 within ESP and AH*
- RFC 2405: *The ESP DES-CBC Cipher Algorithm With Explicit IV*
- RFC 2144: *The CAST-128 Encryption Algorithm*
- RFC 2367: *PF_KEY Key Management API, Version 2*

- RFC 2410: *The NULL Encryption Algorithm and Its Use With IPsec*
- RFC 2451: *The ESP CBC-Mode Cipher Algorithms*
- RFC 2857: *HMAC-RIPE-MD-160-96 within ESP and AH*
- RFC 3566: *The AES-XCBC-MAC-96 Algorithm and Its Use With IPsec*
- RFC 3602: *The AES-CBC Cipher Algorithm and Its Use With IPsec*
- RFC 3686: *Using Advanced Encryption Standard (AES) Counter Mode With IPsec Encapsulating Security Payload (ESP)*
- RFC 4308: *Cryptographic Suites for IPsec*
- RFC 4835: *Cryptographic Algorithm Implementation Requirements for Encapsulating Security Payload (ESP) and Authentication Header (AH)*
- RFC 4868: *Using HMAC-SHA-256, HMAC-SHA-384, and HMAC-SHA-512 with IPsec*

5. Additional Documentation

Numerous additional sources of documentation and background reading about IPsec and related technologies are available.

Wind River Documentation

The following Wind River documents provide information about other products related to Wind River IPsec:

- *Wind River IPsec API Reference*
- VxWorks 7 IKE Programmer's Guide
- VxWorks 7 Cryptography Libraries Programmer's Guide
- *Wind River RADIUS and Diameter for VxWorks 7 Programmer's Guide*
- *Wind River EAP Programmer's for VxWorks 7 Guide*

OpenBSD

Licensing and other information about OpenBSD is available at the OpenBSD Project Web site:

http://www.openbsd.org/policy.html

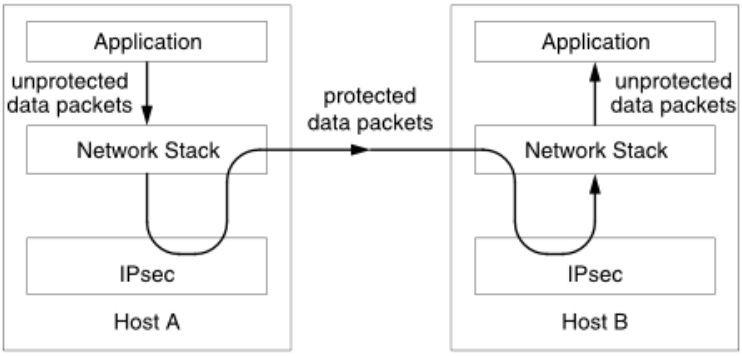
2. TECHNICAL BACKGROUND

1. How IPsec Works

Wind River IPsec implements the requirements of RFC 4301: Security Architecture for the Internet Protocol.

It also supports other RFCs listed in RFCs on page 2. IP security (IPsec) is a standardized framework for securing IP communications between network peers. It does so by authenticating or encrypting each IP packet using protocols, algorithms, and keying material established by security associations (SAs).

Figure 2. IPsec Data Flow



The diagram above shows how data packets pass from Host A to Host B. When a Host A application first sends a packet to Host B through the IPsec-enabled network stack, IPsec intercepts the packet and applies a protection suite, then returns the packet to the stack. Host B's instance of the stack receives the packet and passes it to IPsec, which performs the reverse protection steps and returns the packet to the stack.

IPsec runs on a host computer or in a security gateway (such as a router or firewall), applying protection to IP packets passing through that machine. IPsec can protect one or more paths between a pair of hosts, between a pair of security gateways, or between a security gateway and a host. Because IPsec applies security at the IP layer, it can protect higher layer protocols such as TCP, UDP, and ICMP. Applications that run above the stack can utilize IPsec without code changes.

IPsec intercepts outbound and inbound IP packets on interfaces bound to the IP stack. It directs each packet according to requirements configured in a security policy database (SPD) by the system administrator. Selector fields in the IP and transport layer headers of the packet are matched against entries in the SPD, with one of the following results:

Match found

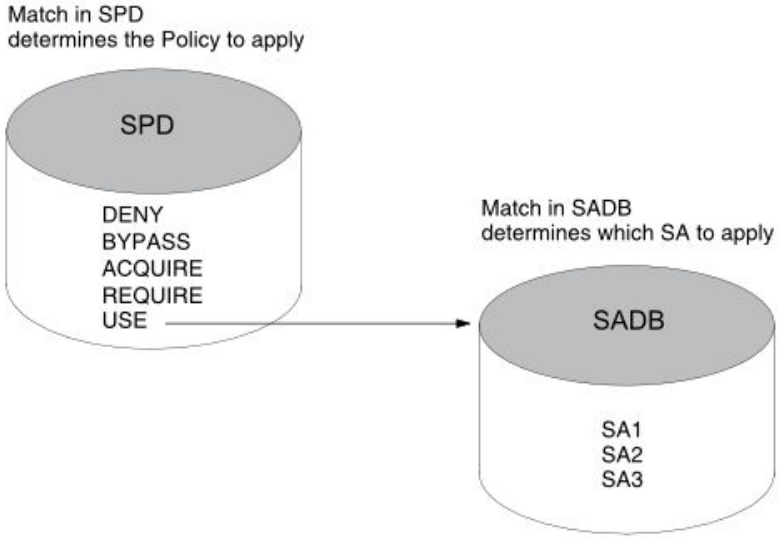
IPsec further processes the packet as described below.

Match not found

IPsec applies its default policy to either discard the packet or allow it to bypass security processing.

An output flow (policy) contains various selectors. Output packets are matched against these selectors to find the first matching policy. If this policy is of USE type, it then points to the SA that should be used (applied), as shown in the figure below. Selectors are used to find an SA, but SAs do not contain any selectors themselves, only the policies do. For more information about SAs, see Security Associations on page 10.

Figure 3. SPD and SADB Relationship



An IPsec flow, or policy, can be one of multiple types: *use*, *require*, *acquire*, *bypass*, or *deny*. For more information about flows, see Flow on page 12.

Bypass flows are associated with a single global SA identified by SPI = 0, security protocol none, and destination address 0.0.0.0.

A *deny* flow silently discards packets without generating any PF_KEY v2 message, SA, or syslog error.

A *require* policy sends an acquire message to any subscribing IKE daemons and discards the packet(s).

An *acquire* policy also sends an acquire packet to any subscribers but lets the packet(s) pass in clear text until SAs are negotiated.

A *use* match in the SPD indicates further processing, which handles the packet using three traffic security protocols, either singly or in combination: the Encapsulating Security Payload (ESP) protocol, the Authentication Header (AH) protocol, and the IP-in-IP (IPIP) encapsulating protocol.

All three protocols make use of security associations (SAs). An SA is a contract between two network peers that governs how the peers protect data passing between them. SAs are unidirectional. Two SAs are required for communication between two peers.

SAs are defined according to the system and security requirements of the organization, applications, and users. They may be created automatically by the system on demand (using IKE) or set up manually by the system administrator. SAs are stored in a security association database (SADB).

The policy matched in the SPD determines which SA in the SADB should be used with a packet.

Match found

IPsec processes the packet as specified by the SA, then returns the packet to the stack. For an outbound packet, the processing may include the addition of an extension header.

Match not found

IPsec applies its default policy to either discard the packet or allow it to bypass security processing.

The AH and ESP protocols use cryptographic algorithms. The IETF specifies a standard set of algorithms in order to promote interoperability on the global Internet. However, IPsec itself is algorithm-independent, enabling different user communities to use preferred algorithm sets. The protocols and algorithms to be employed in a given context are SA parameters.

The ESP and AH protocols have two modes: transport mode and tunnel mode. In transport mode, they primarily protect upper layer protocols, such as TCP, UDP, and ICMP. In tunnel mode, they protect IP tunnels by including an outer header that carries the

addresses of IPsec peers or security gateways. The inner header contains the ultimate source and destination addresses. For host-to-host communication, either transport or tunnel mode can be used, but for communication with a security gateway, only tunnel mode can be used.

IPsec uses shared keys for authentication and encryption/decryption. It relies on an automated or manual key management mechanism to establish these keys, code them in SAs, and distribute them to the computers that require them. The automated mechanism selected by the IETF for use with IPsec is Internet Key Exchange (IKE), as used under the IPsec domain of interpretation.

ESP Protocol on page 6 The ESP protocol is detailed in RFC 4303: IP Encapsulating Security Payload (ESP).
AH Protocol on page 8 The AH protocol is detailed in RFC 4302: IP Authentication Header.
IPIP Protocol on page 9 The IPIP protocol is used to bridge portions of networks that have dissimilar capabilities or protocols.
Security Associations on page 10 An SA is a contract between two network peers, negotiated by IKE or set manually, that governs how the peers protect data passing between them.
SA Design on page 11 The security services specified by an SA include the protocol (AH or ESP) and its optional services, the mode (transport or tunnel), and the endpoints.
Combining SAs on page 11 An individual SA provides protection using a single security protocol, either AH or ESP, but sometimes a security policy calls for a combination of services for a particular traffic flow.
SA and Key Management on page 12 The services that IPsec can provide depend somewhat on the SA and key management techniques employed.
Flow on page 12A flow is a set of packet selectors consisting of a source IP address, a destination IP address, and optionally, an IP protocol, UDP/TCP source port and destination port, ICMP type and code, Mobility Header type, and DSCP value.
ESN on page 13ESN functionality provides support for extended anti-replay protection for both AH and ESP IPsec security protocols.
Anti-replay Window on page 13ESN for IPsec involves anti-replay window calculation and sequence number verification.
Traffic Flow Confidentiality (TFC) on page 14 Traffic Flow Confidentiality (TFC) requires insertion of random padding in every IPsec ESP packet and insertion of dummy packets into the traffic flow.

1.1. ESP Protocol

The ESP protocol is detailed in RFC 4303: IP Encapsulating Security Payload (ESP).

ESP can provide confidentiality (encryption) and limited traffic flow confidentiality (TFC). Alternatively, it can provide data origin authentication, connectionless data integrity, and replay protection. Only one of these two sets of capabilities can be employed at a time.

ESP can perform the same operations as AH, except that it does not protect the outer IP header. In addition, however, it can provide confidentiality by encrypting the payload data using a secret encryption key. While only those who know this key can decrypt the data, it is vulnerable to decryption if the key-length is too short and an attacker has a powerful enough CPU.

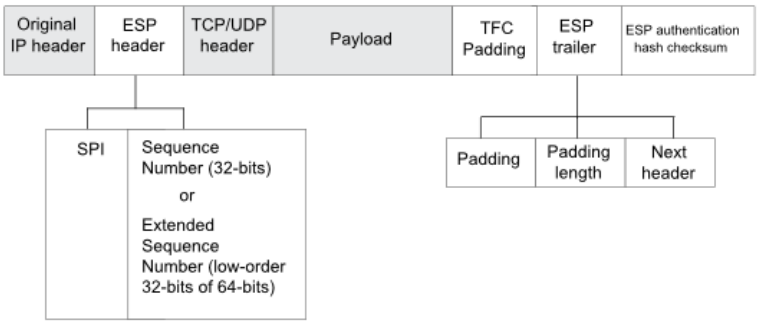
The ESP header contains a 32-bit packet sequence number or extended sequence number (ESN) that is used for replay protection. It gives protection against replay of the same packet twice. For a given anti-replay window size, if the receiver sees the same sequence number again, the packet is rejected without further processing. A 64-bit extended sequence number (ESN) can be negotiated using IKEv2 or set via the keyadm command. For information about keyadm, see keyadm Command Reference on page 35. Wind River IPsec uses IKEv2 for SA management; ESN is enabled by default.

ESP can provide authentication by digitally signing the packet. The main difference between ESP and AH authentication is the extent of coverage. ESP authentication covers the original IP header in tunnel mode only, whereas AH authentication always covers the entire original packet.

ESP provides Traffic Flow Confidentiality (TFC). TFC requires insertion of random padding in every IPsec ESP packet and insertion of dummy packets into the traffic flow.This is done for masking the flow characteristics from eavesdroppers. TFC support must be explicitly negotiated per SA and is not enabled by default. TFC generally is effective only if ESP conceals the ultimate source and destination addresses of correspondents, that is, in tunnel mode between security gateways, and only if sufficient traffic flows between IPsec peers (naturally or as a result of masking traffic) to conceal the characteristics of individual subscriber traffic flows.

The following diagram shows the format of an ESP-protected packet.

Figure 4. Format of ESP-Protected Packet



Some fields of the ESP-protected packet are described below:

Security Parameters Index

The SPI, in conjunction with the destination address and the IPsec protocol (AH or ESP), identifies the SA to be employed for the current packet.

Sequence Number/Extended Sequence Number (ESN)

This is a monotonically increasing counter that uniquely identifies every packet. The sender transmits the counter and the receiver monitors its value. When the counter reaches its maximum value of 232-1 (when using 32-bit Sequence Numbers), or 264-1 (when using 64-bit Extended Sequence Numbers), it returns to zero. The current SA becomes invalid and the peers must negotiate a new SA. Likewise, if the counter value falls below the lower limit of the configured replay window, as in a replay attack, the current SA becomes invalid.

Payload

If ESP encrypts the payload, the Initialization Vector (IV) for the encryption algorithm is typically stored at the beginning of the payload field. For example, if the algorithm is Data Encryption Standard (DES) with Cipher Block Chaining (CBC), the first eight bytes of the payload contain the IV.

TFC Padding

The payload data field has a substructure that depends on the choice of encryption algorithm and mode, and on the use of TFC padding.

An optional field within the payload data allows for the insertion of padding for TFC after the payload data and before the ESP trailer.

For proper discarding of TFC padding, it is required that the payload length can be determined from the inner payload protocol header; for example, when the next_header field in the ESP trailer indicates UDP, IPv4, or IPV6.

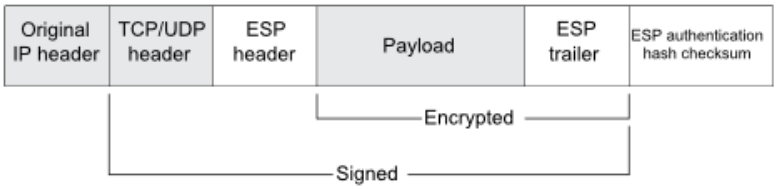
Authentication Data

ESP can provide data integrity by means of a digital signature that includes a cryptographic checksum called the Integrity Check Value (ICV). The sender computes the ICV by inputting the original packet and an authentication key to an HMAC algorithm. To authenticate the packet, the receiver recalculates the ICV and compares it to the ICV in the packet.

ESP Transport Mode

In transport mode, the ESP header occupies a position between the IP header and the payload. The digital signature in the ESP trailer covers all but the IP header. Transport mode can only be used for host-to-host communication.

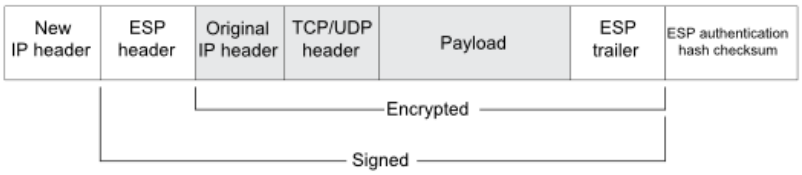
Figure 5. ESP Header Placement in Transport Mode



ESP Tunnel Mode

In tunnel mode, the ESP header precedes the original IP header. Tunnel mode adds a new IP header, encapsulating the original packet in a new packet that can be transported by another network. The digital signature in the ESP trailer covers all but the new IP header.

Figure 6. ESP Header Placement in Tunnel Mode



Parent topic: How IPsec Works on page 4

1.2. AH Protocol

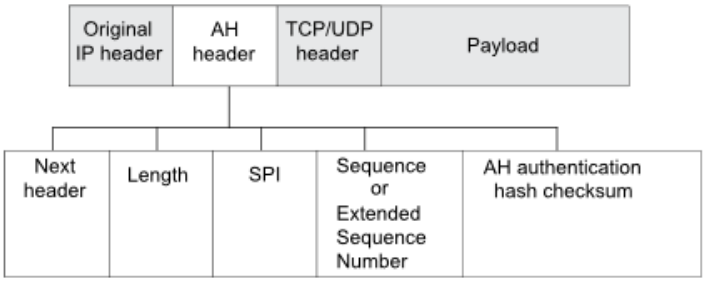
The AH protocol is detailed in RFC 4302: IP Authentication Header.

AH provides data origin authentication, connectionless data integrity, and optional replay protection. AH does not provide confidentiality because it does not perform encryption. The data is readable, but is protected from modification by an integrity checksum. If required, IPsec can further protect AH packets using ESP.

The AH header contains a packet sequence number or extended sequence number (ESN) which alerts the receiver to a possible replay attack, or the sending of the same packet twice.

AH digitally signs the packet by computing a hash from the packet's payload data, some of the IP header data, and a secret authentication key. IPsec then sends this hash together with the original data. The receiver performs the same computation. Identical hash values mean there was no tampering with the data (integrity), the address information (authenticity) or the packet sequence number (replay protection). AH authentication covers the entire packet, including the IP header.

Figure 7. Format of AH-Protected Packet



The diagram above shows the format of an AH-protected packet.

Some fields of the AH-protected packet are described below:

Security Parameters Index

The SPI, in conjunction with the destination address and the IPsec protocol (AH or ESP), identifies the SA to be employed for the current packet.

Sequence Number/Extended Sequence Number

This is a monotonically increasing counter that uniquely identifies every packet. The sender transmits the counter and the receiver monitors its value. When the counter reaches its maximum value of 232-1 (when using 32-bit Sequence Numbers), or 264-1 (when using 64-bit Extended Sequence Numbers), it returns to zero. The current SA becomes invalid and the peers must negotiate a new SA. If the counter value repeats, as in a replay attack, the packet is dropped and a message is logged. With ESN, the size of the sequence number is increased to 64 bits. The security protocol payloads do not need modification because only the lower order 32 bits of the ESN are copied onto the payloads. Use of ESN is negotiated by the IKEv2 SA management protocol and is on by default.

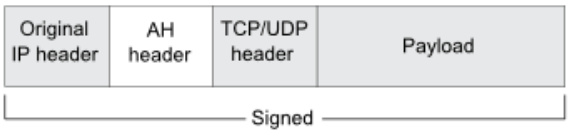
Authentication Data

AH provides data integrity by means of a digital signature that includes a cryptographic checksum called the Integrity Check Value (ICV). The sender computes the ICV by inputting the original packet and an authentication key to an HMAC algorithm. To authenticate the packet, the receiver recalculates the ICV and compares it to the ICV in the packet.

AH Transport Mode

In transport mode, the AH header occupies a position between the IP header and the payload. The digital signature in the AH header covers the entire packet. Transport mode can only be used for host-to-host communication.

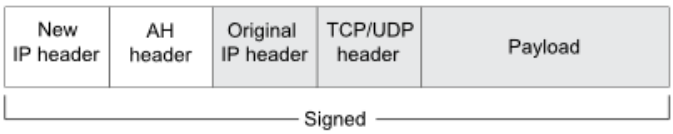
Figure 8. AH Header Position in Transport Mode



AH Tunnel Mode

In tunnel mode, the AH header precedes the original IP header. Tunnel mode adds a new IP header, encapsulating the original packet in a new packet that can be transported by another network. The digital signature in the AH header covers the entire packet.

Figure 9. AH Header Position in Tunnel Mode



Parent topic: How IPsec Works on page 4

1.3. IPIP Protocol

The IPIP protocol is used to bridge portions of networks that have dissimilar capabilities or protocols.

It works by wrapping an IP packet in another standard IP header. The protocol inserts no header items of its own. When the source and destination addresses in the new header are set so as to be accepted by other networks, the resulting packet can freely traverse those networks. The original packet is said to “tunnel” through those networks.

⚠ Warning: Tunnels can introduce security risks, such as the bypassing of firewalls. You should only enable IPIP under clearly defined conditions.

The diagram below shows the format of an IPIP packet.

Figure 10. IPIP Packet Format

New IP header	Tunnel headers	Original IP header	TCP/UDP header	Payload

For detailed information, see RFC 1853: *IP in IP Tunneling*.

Some fields of the IPIP-protected packet are described below:

New IP header

The source and destination fields of the new IP header identify the endpoints of the tunnel.

Tunnel headers

This optional field contains other headers specific to the path, such as security headers specific to the tunnel configuration.

Original IP header

The source and destination fields of the original IP header identify the original sender and recipient of the packet.

Parent topic: How IPsec Works on page 4

1.4. Security Associations

An SA is a contract between two network peers, negotiated by IKE or set manually, that governs how the peers protect data passing between them.

An SA applies to packets of a specific traffic type passing between specific ports on hosts at specific addresses. It determines the security protocol (AH or ESP), protocol mode, and protocol options to be used by IPsec for those packets. It is uniquely identified by a 4-tuple consisting of a Security Parameter Index (SPI), the source address, the destination address, and the protocol.

An SA may specify the use of AH or ESP, but not both. If both AH and ESP are required, two or more SAs must be created and grouped together. An AH SA specifies the authentication key and the hash algorithm to be used. An ESP SA specifies the encryption key, the encryption algorithm to be used, and, optionally, the authentication key and the hash algorithm to be used.

For typical bidirectional communication between two hosts or between two security gateways, two SAs are required, one for each direction.

Currently, SA management mechanisms in IPsec are defined only for unicast SAs. In the following discussion, SAs are described in the context of point-to-point communication.

Transport Mode SAs

A transport-mode SA is an association between two hosts. In IPv4, the security protocol header appears immediately after the IP header and any options, and before any higher layer protocols such TCP or UDP. In IPv6, the security protocol header appears after the base IP header and extensions, but may appear before or after destination options, and before higher-layer protocols.

In the case of ESP, a transport-mode SA only provides protection for higher-layer protocols, not for the IP header or any extension headers preceding the ESP header.

In the case of AH, protection extends to parts of the IP header and parts of certain extension headers.

Tunnel Mode SAs

A tunnel-mode SA is essentially an SA applied to an IP tunnel. Two hosts may optionally establish a tunnel-mode SA between themselves, unless either end is a security gateway, in which case the SA is required to be tunnel-mode. This requirement arises when multiple paths through different security gateways exist to the same destination. (When traffic is destined for a security gateway itself, as in the case of SNMP commands, the gateway acts as a host, and transport mode is allowed.)

A packet processed by a tunnel-mode SA has an outer IP header that specifies the IPsec processing destination, and an inner IP header that specifies the apparent ultimate destination. The security protocol header appears after the outer header and before the inner header.

If AH is employed in tunnel mode, portions of the outer IP header are protected (as in transport mode), as well as the entire tunneled IP packet, that is, the inner IP header and any higher layer protocols.

If ESP is employed, only the tunneled packet is protected, not the outer IP header.

Parent topic: How IPsec Works on page 4

1.5. SA Design

The security services specified by an SA include the protocol (AH or ESP) and its optional services, the mode (transport or tunnel), and the endpoints.

AH provides data origin authentication and connectionless integrity for IP packets (together referred to as authentication). The precision of the authentication depends on the granularity of the SA, which depends on the selectors. AH also offers replay protection at the discretion of the receiver, to help counter denial-of-service attacks. AH is an appropriate protocol to employ when confidentiality is not required or is not permitted due to government restrictions on the use of encryption. AH also provides authentication for selected portions of the IP header, which may be necessary in some cases; for example, if the integrity of an IPv4 option or IPv6 extension header must be protected (except for the non-predictable but mutable parts of the IP header.)

ESP optionally provides traffic confidentiality, the strength of which depends partly on the encryption algorithm employed. ESP optionally provides authentication, as described above for AH. If authentication is negotiated for an ESP SA, the receiver may also elect to enforce replay protection, as described for AH. AH provides stronger authentication than ESP because IP headers outside the ESP header are not protected.

If only the upper layer protocols need to be authenticated, then ESP authentication is an appropriate choice and is more space-efficient than using AH to encapsulate ESP. Note that either confidentiality or authentication must be selected. If confidentiality is selected, then tunnel-mode SA between two security gateways can offer partial traffic flow confidentiality. The use of tunnel mode allows the inner IP headers to be encrypted, concealing the identities of the (ultimate) traffic source and destination. Moreover, ESP payload padding can be used to hide the size of the packets, further concealing the external characteristics of the traffic. Traffic flow confidentiality (TFC) can also be applied. TFC support must be explicitly negotiated per SA and is not enabled by default.

Note that fine-granularity SAs are generally more vulnerable to traffic analysis than coarse-granularity ones that carry traffic from many subscribers.

Parent topic: How IPsec Works on page 4

1.6. Combining SAs

An individual SA provides protection using a single security protocol, either AH or ESP, but sometimes a security policy calls for a combination of services for a particular traffic flow.

In such cases it is necessary to employ multiple SAs. The term SA bundle refers to a sequence of SAs through which traffic must be processed to satisfy a security policy.

WNDRVR

The policy defines the order of the sequence. Note that the SAs that comprise a bundle may terminate at different endpoints. For example, one SA may extend between a mobile host and a security gateway and a second, nested SA may extend to a host behind the gateway.

SAs may be combined in two ways: transport adjacency and iterated tunneling. Transport adjacency refers to applying more than one protocol to the same IP packet, without invoking tunneling. Iterated tunneling refers to applying multiple layers of security by nested tunneling.

Parent topic: How IPsec Works on page 4

1.7. SA and Key Management

The services that IPsec can provide depend somewhat on the SA and key management techniques employed.

For example, replay protection services require automated SA management. Data origin authentication is limited by the extent to which the authentication key is shared among the possible data origins.

Wind River IPsec includes facilities for automated and manual management of SAs and cryptographic keys.

Automated Management

The widespread deployment of IPsec requires an IETF standard for a scalable, automated SA and key management protocol. The protocol selected by the IETF for use with IPsec is Internet Key Exchange (IKE), as used under the IPsec domain of interpretation. IKE enables on-demand creation of SAs for user- and session- oriented keying, including the process of "rekeying" an SA, which creates a new SA with a new SPI. Automated management is also required to enable use of the anti-replay features of AH and ESP.

Wind River IKE implements the IKE protocols (versions 1 and 2) according to the IETF requirements. For more information on Wind River IKE, see the *VxWorks 7 IKE Programmer's Guide*. Wind River IPsec communicates with Wind River IKE by means of the PF_KEY v2 key management interface.

Manual Management

In manual management, an administrator configures each host with keying material and SA parameters. Manual management is practical in small, static environments. It may also be appropriate when only certain communications need to be secured. It often employs statically configured symmetric keys, though other options exist.

For example, a company could create a VPN using IPsec in security gateways at several sites. If the number of sites is small and they are all in a single administrative domain, manual management is likely to be a feasible. Each gateway could protect traffic within the organization, using a manually configured key. Traffic to and from outside destinations would not be protected. Another suitable case for manual management might be a small organization that uses IPsec for internal communications.

Wind River IPsec includes an extended PFKEY version2 C-API, as well as the keyadm shell command, for manual SA management. For more information on shell commands and the PFKEY API functions, see ipsecctrl Command Reference on page 33, keyadm Command Reference on page 35 and the IPSec API reference.

Parent topic: How IPsec Works on page 4

1.8. Flow

A flow is a set of packet selectors consisting of a source IP address, a destination IP address, and optionally, an IP protocol, UDP/TCP source port and destination port, ICMP type and code, Mobility Header type, and DSCP value.

In Wind River IPsec, flows describe traffic for the purpose of determining the security processing to be applied to the traffic. Wind River IPsec specifies a flow when it sends an SA request to its key management task.

Flows of the following types identify specific SAs:

input

An *input* flow determines the handling of incoming packets. Possible actions are acquire, require, bypass, deny and use. use means that a packet must have been processed successfully by IPSec in order to be accepted by the target. In other words, incoming cleartext packets are discarded if there is a matching use input flow. If input flows are missing, incoming cleartext packets are accepted as well as IPSec processed packets. Wind River IPsec checks input flows after traffic has been processed by IPSec. Note that input flows are independent of incoming SAs, that is, they do not point to a specific SA (this includes the use type).

output

An *output* flow determines which packets are routed by a specific outgoing SA. When output packet filtering is enabled and outgoing traffic matches an output flow, the flow determines the SA to be used. Wind River IPsec checks output flows after traffic passes through NAT and the firewall.

The following types of flows are not associated with a specific SA:

acquire

An *acquire* flow generates a PF_KEY v2 Acquire message to a key management task (to the first registered listener), requesting it to generate an SA. This type of flow accepts and sends the outgoing packets in plaintext until the SA has been negotiated. Wind River IKE does not add acquire flows to the SADB.

require

A *require* flow generates a PF_KEY v2 Require message to a key management task (to the first registered listener), requesting it to generate an SA. This type of flow discards the packets until the SA has been negotiated. Wind River IKE does not add require flows to the SADB.

bypass

A *bypass* flow specifies packets that are allowed to bypass IPsec processing. All bypass flows are associated with a single global SA identified by SPI = 0, security protocol none, and destination address 0.0.0.0. (Some packets have to bypass IPsec so that key management tasks can communicate with each other. Wind River IKE automatically adds a bypass flow to the SADB for the address and port of its own host.

deny

A *deny* flow silently discards packets without generating any PF_KEY v2 message, SA, or syslog error.

To manually create and modify flows, use the PFKEY programming API or keyadm shell command.

Parent topic: How IPsec Works on page 4

1.9. ESN

ESN functionality provides support for extended anti-replay protection for both AH and ESP IPsec security protocols.

The purpose of ESN is to provide efficient anti-replay support for high-speed IPsec implementations.

Parent topic: How IPsec Works on page 4

1.10. Anti-replay Window

ESN for IPsec involves anti-replay window calculation and sequence number verification.

The original implementation of the anti-replay mechanism in IPsec utilizes 32-bit sequence numbers for protection against replay. ESN increases the size of the sequence number to 64-bits. The security protocol payloads do not need modification because only the lower order 32-bits of the ESN are copied into the payloads.

ESN can be enabled per-SA using the keyadm shell command option -esn when adding an SA.

For more information, see keyadm add on page 35. For each SA, the ESN feature must be negotiated automatically using the SA management entity (IKE) or it must be configured manually.

Parent topic: How IPsec Works on page 4

1.11. Traffic Flow Confidentiality (TFC)

Traffic Flow Confidentiality (TFC) requires insertion of random padding in every IPsec ESP packet and insertion of dummy packets into the traffic flow.

TFC randomizes the characteristics of IPsec traffic flows with the dummy packets. The randomness imparted depends on the efficiency of the random number generator. This is done to mask the flow characteristics from eavesdroppers.

You must specify proper input and output policies so that dummy packets can be sent and received between the endpoints of the SA. That is, for each SA between an IP source address and an IP destination address, there should be corresponding IPsec flows (IPsec policies) defined between the IP source address and the IP destination address to allow the dummy packets which are generated to be sent. If the flows are not created properly, the dummy packets are dropped and an error message indicating a missing policy is logged.

For each SA, the TFC feature must be negotiated automatically using the SA management entity (IKE) or it must be configured manually.

Parent topic: How IPsec Works on page 4

2. PF_KEY v2 Key Management Interface

Learn about the PF_KEY v2 interface described in RFC 2367: PF_KEY Key Management API, Version 2.

For detailed information, see this and other RFCs listed in RFCs on page 2.

PF_KEY v2 is a generic key management API definition. Its purpose is to provide an OS-independent interface between key management applications, such as Wind River IKE, and OS-specific key engines. The primary logical element of a key engine is a security association database (SADB). PF_KEY v2 uses a defined set of messages to insert SA information into and retrieve information from the SADB.

When IPsec processes an IP packet according to policy, it may request an SA from the key engine. If the key engine has a suitable SA, IPsec proceeds to use it. If the key engine has no suitable SA, but a key management application has previously indicated, by means of the **SADB_REGISTER** message, that such SAs can be obtained, then the key engine requests the creation of a new SA, by means of an **SADB_ACQUIRE** message. The key management application adds the new SA to the key engine for future use.

There are two basic ways to add a new SA to the key engine. The simplest is for the key management application to send a single **SADB_ADD** message, containing all of the SA information, to the key engine. The second approach is for the application to request a Security Parameter Index (SPI) from the IPsec task, by means of the **SADB_GETSPI** message, and then send an **SADB_UPDATE** message with the complete SA data.

Wind River IPsec uses the PF_KEY v2 key management API to request its key engine, the SADB, to add, configure, and delete SAs.

To do so, Wind River IPsec adds the PF_KEY domain to the TCP/IP stack. This provides a socket API to the SADB and enables the BSD socket functions (socket( ), read( ), write( ), and so on) to be used to perform SA operations.

In addition, Wind River IPsec implements the majority of the OpenBSD 2.7 PF_KEY extensions, enabling interoperation with any PF_KEY v2-compliant implementation of IKE that also supports the OpenBSD extensions.

Wind River IPsec also implements a few Linux extensions such as **SADB_X_SPDADD**, **SADB_X_SPDUPDATE**, **SADB_X_SPDDELETE** and **SADB_X_SPDFLUSH**, as well as some proprietary extensions, for example, to list and dump flows.

The following header file defines the PF_KEY v2 interface and the OpenBSD extensions:

installDir **/vxworks-7/pkgs_v2/net/ipnet/ipsecike-***x.x.x.x* **/ipsec/h/ipipsec_pfkeyv2.h**

For PF_KEY v2 programming examples, see the following application code:

installDir **/vxworks-7/pkgs_v2/net/ipnet/ipsecike-***x.x.x.x* **/ipsec/src/ipipsec_cmd_keyadm.c**

3. MIB Interface

Wind River IPsec can optionally collect and store statistics information and record it in variables described by the IPsec Monitor MIB.

\Applications can therefore implement the mandatory IPsec Monitor MIB objects described in **draftietf-ipsec-doi-tc-mib-03.txt** and **draft-ietf-ipsec-monitor-mib-03.txt**.

4. Cryptographic Suites for IPsec

Wind River IPsec complies with RFC 4308: Cryptographic Suites for IPsec.

This RFC specifies suites of attributes and algorithms that can be used to administer IPsec in manual keying mode with either IKEv1 or IKEv2.

Wind River IPsec implements the VPN-A and VPN-B suites as described in RFC 4308. These cryptographic suites are for use in IPsec virtual private networks. For configuration information and examples, see Configuring Cryptographic Suites for VPNs on page 26 and the *VxWorks 7 IKE Programmer's Guide*.

5. Cryptographic Support

Build-time configuration of Wind River IPsec includes the selection of a cryptographic provider module.

At run time, configuration of the IPsec task includes the selection of cryptographic algorithms to be used in SAs. For an algorithm to be usable, both Wind River IPsec and the provider module must have the necessary capabilities.

Cryptographic Provider Modules

Wind River IPsec obtains cryptographic services by passing requests to a cryptographic provider module, which handles communications between IPsec and the required cryptographic algorithms. The algorithms themselves can be implemented in either software or hardware.

If desired, you can add support for new algorithms. To add an algorithm, code its name, identifier, key length, and other required information in the mapping table in:

installDir **/vxworks-7/pkgs_v2/net/ipnet/ipsecike-***x.x.x.x* **/ipsec/src/ipipsec_crypt.c**

Wind River IPsec can then use a hardware or software implementation of the algorithm.

Wind River IPsec Cryptographic Technology Support on page 16

5.1. Wind River IPsec Cryptographic Technology Support

Symmetric Cipher Algorithm Support

The following table lists the Symmetric Cipher algorithms supported by Wind River IPsec.

Table 1. Symmetric Cipher Algorithms

Symmetric Cipher Algorithm Type	Standards
AES128-CBC	NIST FIPS 197
AES128-CTR	NIST FIPS 197
AES192-CBC	NIST FIPS 197
AES256-CBC	NIST FIPS 197
AES256-CTR	NIST FIPS 197
3DES-56-CBC (DES-56-CBC)	NIST FIPS 46-3
3DES-168-CBC	NIST FIPS 46-3
Blowfish-128	
CAST-128	RFC 2144
NULL	N/A

Hash and Message Digest Support

The following table lists the Hash and Message Digest algorithms supported by Wind River IPsec.

Table 2. Hash and Message Digest Algorithms

Hash and Message Digest Type	Standards
RIPEMD-160	
MD5-128	RFC 1321
SHA-256	NIST FIPS 180-4

Hash and Message Digest Type	Standards
SHA-384	NIST FIPS 180-4
SHA-512	NIST FIPS 180-4

Message Authentication Code Support

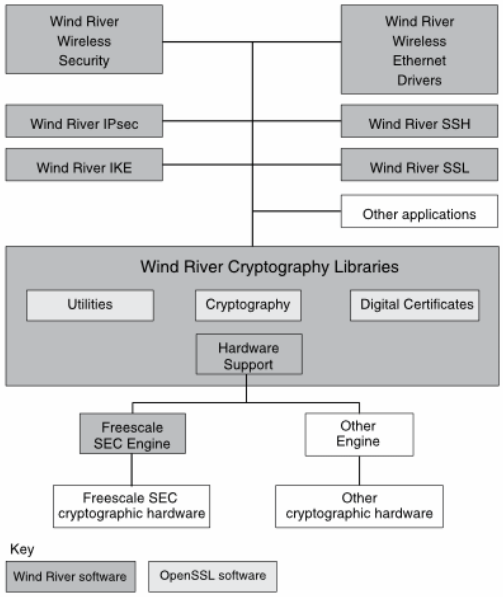
The following table lists the Message Authentication Code algorithms supported by Wind River IPsec.

Table 3. Message Authentication Code Algorithms

Message Authentication Code Type	Standards
AES-XCBC-MAC- 96 (128 bit support)	NIST FIPS 197
AES-XCBC-MAC-128	NIST FIPS 197
RIPEMD-160	
MD5-128	RFC 1321
HMAC-SHA-256	RFC 4868
HMAC-SHA-384	RFC 4868
HMAC-SHA-512	RFC 4868

The relationships between Wind River IPsec, Wind River Cryptography Libraries, and other Wind River products are expressed in the following diagram.

Figure 11. Wind River Product Relationships



Parent topic: Cryptographic Support on page 15

6. Logging and Statistics

Wind River IPsec logs events, such as the addition and removal of SAs, to the syslog task.

The component for syslog is **INCLUDE_IPCOM_SYSLOGD_CMD**. Messages output to syslog are printed by default on a debug port, but the logging function can also write the log entries to a file or a UDP socket.

The amount of information recorded depends on the debug level. You can set the debug level using the syslog shell command. For example, the command:

```
syslog priority ipipsec Emerg
```

would log all IPsec related Emergency level messages to syslog.

Synopsis:

```
syslog echo prio message
syslog list
syslog priority facility prio
syslog log file [logfile]
```

Where facility is **ipipsec** and prio is the debug priority level of the specified message. Options include:

- Emerg
- Crit
- Error
- Warning
- Notice
- Info
- Debug
- Debug2

The table below lists the syslog message priority levels.

Table 1. Syslog Message Priority Levels

Level Name	No.	Workbench Configuration Parameter	Priority
EMERGENCY	0	IPCOM_LOG_EMERG	Highest Priority
CRITICAL	1	IPCOM_LOG_CRIT	
ERROR	2	IPCOM_LOG_ERR	Default log output level at run time, as defined by IPCOM_SYSLOGD_DEFAULT_PRIORITY.

Level Name	No.	Workbench Configuration Parameter	Priority
WARNING	3	IPCOM_LOG_WARNING	This level and higher compiled in for normal builds.
NOTICE	4	IPCOM_LOG_NOTICE	
INFO	5	IPCOM_LOG_INFO	
DEBUG	6	IPCOM_LOG_DEBUG	Lowest priority. This level and higher compiled in for debug builds.
DEBUG 2	7	IPCOM_LOG_DEBUG2	Default level when debug flag is used.

For complete information on syslog, see the Wind River Network Stack documentation set.

Byte counts and other statistics such as timer values are recorded in variables described by the IPsec Monitor MIB. The ipsecctrl mib shell command prints the MIB variables to the console.

3. CONFIGURING AND BUILDING WIND RIVER IPSEC

1. Configuring a VSB

Wind River IPsec is shipped as source code that you must compile before it can be used with either kernel or real-time process (RTP) applications.

To compile your source code you must create a VxWorks source build (VSB) project as directed in the *VxWorks 7 Configuration and Build Guide*, which provides the following information:

- Instructions for creating a VSB project with Workbench or from the command line
- Instructions on how to configure and compile the source code libraries used to develop VxWorks kernel and user-space applications
- A description of the configuration options for libraries used to develop the VxWorks operating system facilities, such as support for multiprocessing, real-time processes, and object management

To configure your VSB project with support for Wind River IPsec, you must include the following component:

IPNET_IPSEC

This is the Wind River IPsec component. Enable it to include IPsec support in your VSB.

2. Configuring a VxWorks Image Project

A VxWorks image project (VIP) is used to create a version of VxWorks that includes components which support features of IPsec.

For information and instructions for building VxWorks, refer to the For instructions, see the *VxWorks 7 Configuration and Build Guide*

See the online *Wind River IPsec Component Reference*, accessible through the Workbench online help, for information and descriptions of the Wind River IPsec components and parameters.

INCLUDE_IP_SECURITY

This is the Wind River IPsec component. Include it to add IPsec support to your VxWorks image project.

For more information, see INCLUDE_IP_SECURITY Configuration Parameters Reference on page 20 and INCLUDE_IP_SECURITY Configuration Parameters Reference on page 20.

INCLUDE_IPSECCTRL_CMD

This is the **ipsecctrl** component. Include this component to provide support for use of the ipsecctrl commands.

INCLUDE_KEYADM_CMD

This is the keyadm shell command component. Include this component to provide support for use of the keyadm commands.

3. INCLUDE_IP_SECURITY Configuration Parameters Reference

The following table lists and describes the INCLUDE_IP_SECURITY on page 20 parameters, which you can set in the VxWorks image project (VIP).

Table 1. INCLUDE_IP_SECURITY Parameters

Parameter Name	Description	Default
IPIPSEC_CONF_ESP_ENABLE	Set to 1 to enable the ESP protocol. Set to 0 to disable it.	1
IPIPSEC_CONF_AH_ENABLE	Set to 1 to enable the AH protocol. Set to 0 to disable it.	1
IPIPSEC_CONF_IPIP_ALLOW	Set to 1 to enable IPIP tunneling. Set to 0 to disable it.	1
IPIPSEC_CONF_ENABLE_OUTFILTER	Set to 1 to enable output filtering. Set to 0 to disable it.	0
IPIPSEC_LARVAL_SA_TMO_SECONDS	Larval SA timeout value in seconds. Sets the timeout for the default larval SA created by the keyadm getspi shell command. (A larval SA is an SA that has not been updated to the mature state.) This timeout determines when a larval SA is automatically released.	600
IPIPSEC_MAX_TFC_DELAY	Set the maximum TFC dummy packet generation delay, in seconds.	10
IPIPSEC_MIN_TFC_DELAY	Set the minimum TFC dummy packet generation delay, in seconds.	5
IPIPSEC_MAX_DUMMY_SIZE	Set the maximum size of the randomly generated dummy packets, in bytes.	500
IPIPSEC_MAX_TFC_PADDING	Set the maximum limit on the padding added to each packet that is processed by IPsec.	255
IPIPSEC_CONF_SA_CACHE_MAXSIZE	Configure maximum SA cachesize.	16
IPIPSEC_CONF_FLOW_CACHE_MAXSIZE	Configure maximum flow cachesize.	16

Note: If you combine IPsec in an SMP system with hardware cryptography offload, you must use the synchronous mode for hardware cryptography. The asynchronous mode for hardware cryptography offload is not supported when using IPsec SMP. For more information, see the *VxWorks 7 Cryptography Libraries Programmer's Guide*.

4. RUN-TIME CONFIGURATION

1. Introduction

Certain configuration operations can be performed while Wind River IPsec is running.

Such operations are of two types: IPsec task configuration and SA administration. Run-time configuration operations can be performed in several ways.

The related topics describe the following methods:

- programming the initialization code on page 22
- running shell commands on page 26
- running an application program on page 27

2. Programming the Initialization Code

The Wind River IPsec initialization code initializes the IPsec task during the VxWorks kernel start up.

The initialization code consists of the following files:

installDir **/vxworks-7/pkgs_v2/net/ipnet/ipsecike-***x.x.x.x* **/ipsec/h/ipipsec_config.h**

installDir **/vxworks-7/pkgs_v2/net/ipnet/ipsecike-***x.x.x.x* **/ipsec/cfg/ipipsec_config.c**

Related topics describe sysvar settings on page 22, parameter defines on page 23, and API function calls on page 25 that you can program into the initialization code to customize the startup configuration of Wind River IPsec.

> Note: If you modify the initialization code, you must recompile Wind River IPsec as instructed in the For instructions, see the *VxWorks 7 Configuration and Build Guide*

Sysvars on page 22 IPsec sysvars are set in the *ipipsec_conf_sysvar* array in **ipipsec_config.c**, and their equivalent Workbench parameters, which are in the VxWorks Integration Project.
Parameter Defines Reference on page 23 Various parameter defines are set in **ipipsec_config.h**.
Configuration Functions on page 25 Wind River IPsec includes configuration functions.

2.1. Sysvars

IPsec sysvars are set in the *ipipsec_conf_sysvar* array in **ipipsec_config.c**, and their equivalent Workbench parameters, which are in the VxWorks Integration Project.

Sysvars are similar to environment variables, with the difference being that sysvars are global settings for the entire system, and environment variables affect local processes and children processes. Sysvars and their equivalent Workbench parameters are listed in the following table. For descriptions of the Workbench parameters, see INCLUDE_IP_SECURITY Configuration Parameters Reference on page 20.

Sysvar and Equivalent Workbench Parameter	Equivalent Workbench parameter
ipipsec.larval_sa_tmo_seconds IPIPSEC_LARVAL_SA_TMO_SECONDS	Larval SA timeout value in seconds. Sets the timeout for the default larval SA created by the keyadm getspi shell command. (A larval SA is an SA that has not been updated to the mature state.) This timeout determines when a larval SA is automatically released. The default value is 600.
ipipsec.pfkey.esn_is_default No equivalent Workbench parameter.	Use the sysvar shell command to modify the replay window.

Parent topic: Programming the Initialization Code on page 22

2.2. Parameter Defines Reference

Various parameter defines are set in **ipipsec_config.h**.

These parameters are not accessible through Workbench. Set these parameters by editing the **ipipsec_config.h** file.

IPIPSEC_USE_AH

Define this parameter to include the AH protocol. Undefine it to exclude AH and reduce the VxWorks kernel size.

IPIPSEC_USE_IPIP

Define this parameter to include the IPIP protocol. Undefine it to exclude IPIP and reduce the VxWorks kernel size.

IPIPSEC_USE_MD5

Define this parameter to include the MD5 authentication algorithm. Undefine it to exclude MD5 and reduce the VxWorks kernel size.

IPIPSEC_USE_SHA

Define this parameter to include the SHA-1 authentication algorithm. Undefine it to exclude SHA-1 and reduce the VxWorks kernel size.

IPIPSEC_USE_SHA256

Define this parameter to include the SHA-256 authentication algorithm. Undefine it to exclude SHA-256 and reduce the VxWorks kernel size.

IPIPSEC_USE_SHA384

Define this parameter to include the SHA-384 authentication algorithm. Undefine it to exclude SHA-384 and reduce the VxWorks kernel size.

IPIPSEC_USE_SHA512

Define this parameter to include the SHA-512 authentication algorithm. Undefine it to exclude SHA-512 and reduce the VxWorks kernel size.

IPIPSEC_USE_RIPEMD

Define this parameter to include the RIPEMD authentication algorithm. Undefine it to exclude RIPEMD and reduce the VxWorks kernel size.

WNDRVR

IPIPSEC_USE_AESMAC

Define this parameter to include the AES-MAC authentication algorithm. Undefine it to exclude AES-MAC and reduce the VxWorks kernel size.

IPIPSEC_USE_DES

Define this parameter to include the DES encryption algorithm. Undefine it to exclude DES and reduce the VxWorks kernel size.

IPIPSEC_USE_3DES

Define this parameter to include the 3DES encryption algorithm. Undefine it to exclude 3DES and reduce the VxWorks kernel size.

IPIPSEC_USE_AES

Define this parameter to include the AES encryption algorithm. Undefine it to exclude AES and reduce the VxWorks kernel size.

IPIPSEC_USE_AESCTR

Define this parameter to include the AES-CTR encryption algorithm. Undefine it to exclude AES-CTR and reduce the VxWorks kernel size.

IPIPSEC_USE_BLOWFISH

Define this parameter to include the Blowfish encryption algorithm. Undefine it to exclude Blowfish and reduce the VxWorks kernel size.

IPIPSEC_USE_CAST

Define this parameter to include the CAST encryption algorithm. Undefine it to exclude CAST and reduce the VxWorks kernel size.

IPIPSEC_USE_ESN

Define this parameter to enable the use of Extended (64-bit) Sequence Numbers (ESN) for AH and ESP and to enable the anti-replay window. Undefine it to use standard 32-bit Sequence Numbers.

Note: When using cryptographic hardware acceleration (**IPHWCRYPTO**), enabling ESN results in a performance penalty because the underlying hardware driver does not support ESN. It is still possible to use replay protection because it is implemented in such a way that it recalculates the hardware digest in software. 32-bit sequence numbers and replay protection can be used without any significant performance degradation.

IPIPSEC_USE_TFC

Define this parameter to enable TFC (Traffic Flow Confidentiality) for ESP. Undefine it to disable TFC for ESP.

IPIPSEC_USE_PMTU_DISCOVERY

Define this parameter to enable path MTU discovery over IPSEC tunnels. Undefine it to disable MTU discovery.

IPIPSEC_USE_BITARRAY

Define this parameter to enable configuration replay windows for ESN for AH and ESP. For this parameter to have any effect, both IPIPSEC_USE_ESN and IPCOM_USE_BITARRAY must be defined.

IPIPSEC_USE_MIB

Define this parameter to include support for the IPsec Monitor MIB variables.

IPIPSEC_USE_STATS

Define this parameter to enable the generation of debug statistics. (IPsec writes the statistics to an internal data structure called *ipipsec.stats* , which can only be listed by a source code debugger.)

IPIPSEC_USE_CTRL_API

Define this parameter to use deprecated CTRL functions.

IPIPSEC_USE_LINUX_PFKEYV2_API

Define this parameter to include API support for the Linux PFKEY messages SADB_X_SPDADD, SADB_X_SPDUPDATE, SADB_X_SPDDELETE and SADB_X_SPDFLUSH used with Linux PFKey applications. Undefine it to exclude Linux PFKEY API support.

IPIPSEC_USE_SYSLOG

Define this parameter to use syslogs.

IPIPSEC_SYSTEM_PRIORITY

Define this parameter to choose what gets logged in the syslogs. You can choose either IPCOM_LOG_DEBUG2 or IPCOM_LOG_WARNING.

IPIPSEC_USE_TRAPS

Define this parameter to enable the use of SNMP traps in the IPsec Monitor MIB.

IPIPSEC_MIN_SEND_TRAP_SECONDS

Minimum number of seconds between traps for a particular SA and trap type. Default is 3 seconds.

IPIPSEC_USE_EVENT_LOG

Define this parameter to enable event logging in IPsec. You must also enable IPIKE.

IPIPSEC_USE_ROHC

Define this parameter to enable header compression inside IPsec as specified in RFC 5858.

IPIPSEC_USE_CRYPTO_SELF_TEST

Define this parameter to enable self test verification of cryptographic algorithms during start up.

Parent topic: Programming the Initialization Code on page 22

2.3. Configuration Functions

Wind River IPsec includes configuration functions.

ipipsec_ctrl( )

Displays MIB statistics, trap, and flow information from the IPsec task.

ipipsec_sactrl( )

Gets SA parameters in the IPsec task.

For reference documentation on these functions, see the IPSec and IKE API online reference. For examples of the types of output generated by these functions, see the ipsecctrl shell command in ipsecctrl Command Reference on page 33. (The shell command is a wrapper for the ipipsec_ctrl( ) and ipipsec_sactrl( ) functions.)

Parent topic: Programming the Initialization Code on page 22

3. Running Shell Commands

The ipsecctrl and keyadm shell commands perform configuration operations:

ipsecctrl

Retrieves information from the IPsec task. It can print input and output flows, list input and output SAs, and get global IPsec variables. (The shell command is a wrapper for the ipipsec_ctrl( ) and ipipsec_sactrl( ) functions.)

keyadm

Has subcommands to add, delete, get, and modify SAs. It can flush and print all SAs and flows. It can also create and attach input and output filter flows to SAs, and group SAs into bundles. (The term flow refers to the combination of IP protocol, source and destination IP addresses, and other packet selectors, for example, source and destination ports affected by an SA.) This command is a wrapper for a PF_KEY socket capable of exchanging KEY messages with the key engine.

You can run these commands at the command prompt on the target device, or you can run them from a shell script. You can store the script on the target and execute it from a local process, or you can store it on an administrative computer that logs into the target (this approach would require a secure network).

For reference documentation on the shell commands, see ipsecctrl Command Reference on page 33 and keyadm Command Reference on page 35.

4. Configuring Cryptographic Suites for VPNs

Wind River IPsec implements the VPN-A and VPN-B suites as described in RFC 4308.

These cryptographic suites are for use in IPsec virtual private networks. The following sections describe how to configure VPN suites for IPsec SAs using the keyadm command. For information on configuring VPN suites for IKE SAs, see the *VxWorks 7 IKE Programmer's Guide*

Configuring the VPN-A Suite

The VPN-A suite is intended to match the currently most commonly used VPN security settings in IKEv1.

RFC 4308 defines the following algorithms for the IPsec SA VPN-A suite:

- Protocol - Encapsulating Security Payload (ESP) [RFC 2406]
- ESP encryption - TripleDES in CBC mode [RFC 2451]
- ESP integrity - HMAC-SHA1-96 [RFC 2404]

IPsec Configuration

Syntax

```
keyadm add -new -enc 3des -auth sha1 -spi <spi value> -dst <sa dst> -src <SA src> -ekey <168 bit e
ncryption-key in hex> -akey <160 bit authentication key in hex>
```

Example

```
keyadm add -enc 3des -auth sha1 -spi 603 -dst 192.168.0.1 -src 192.168.0.2 -ekey b0e39876daf6d4ee5
aaa895a4733bd8afdead83a2a8aa480 -akey 8317954752bf255b5321fe19e54a7f8d83179547
```

Configuring the VPN-B Suite

The VPN-B suite comprises settings that many expect to be the commonly used VPN security settings within a few years.

RFC 4308 defines the following algorithms for the IPsec SA VPN-B suite:

- Protocol - Encapsulating Security Payload (ESP) [RFC2406]
- ESP encryption - AES with 128-bit keys in CBC mode [AES-CBC]
- ESP integrity - AES-XCBC-MAC-96 [AES-XCBC-MAC]

IPSec Configuration Syntax:

```
keyadm add -new -enc aes -auth aesmac -spi <spi value> -dst <sa dst> -src <SA src> -ekey <128 bit
encryption-key in hex> -akey <128 bit authentication key in hex>
```

Example:

```
keyadm add -enc aes -auth aesmac -spi 603 -dst 192.168.0.1 -src 192.168.0.2 -ekey 8317954752bf255b
5321fe19e54a7f8d -akey 8317954752bf255b5321fe19e54a7777
```

Running an Application Program on page 27 You can write an application program that issues PF_KEY messages to perform SA administration while Wind River IPsec is running.

4.1. Running an Application Program

You can write an application program that issues PF_KEY messages to perform SA administration while Wind River IPsec is running.

For reference documentation on the ipipsec_sactrl( ) function, see the IPSec and IKE API reference.

Parent topic: Configuring Cryptographic Suites for VPNs on page 26

5. EXAMPLE IMPLEMENTATION

1. Example Configuration Program

Examples demonstrate the setup of tunnel and transport connections using the ESP, AH, and IPIP protocols.

These examples demonstrating the setup of tunnel and transport connections using the ESP, AH, and IPIP protocols.

Wind River IPsec includes the following example configuration program:

installDir **/vxworks-7/pkgs_v2/net/ipnet/ipsecike-***x.x.x.x* **/ipsec/src/ipipsec_example.c**

By defining parameters at the top of the program, you can configure Wind River IPsec to process outgoing packets in the following ways:

- ESP with or without IPIP
- AH with or without IPIP
- ESP and AH, with or without IPIP

The program runs on both of the targets shown in the diagram below. It executes keyadm shell commands to add flows, SAs, and SA groups to the target's SADB. Running the program on Target 1 enables Target 1 to send protected packets to Target 2 and to receive protected packets from Target 2. Running the program on Target 2 gives the corresponding capabilities to Target 2.

You must include the following components in your VxWorks image project to provide support for the examples used here:

INCLUDE_IPSECCTRL_CMD

This is the ipsecctrl component. Include this component to provide support for use of the ipsecctrl commands.

INCLUDE_KEYADM_CMD

This is the keyadm shell command component. Include this component to provide support for use of the keyadm commands.

Figure 12. Network for Example Configuration Program

2. Launching the IP_COM ipipsec_config Process to Support keyadm Commands

The ipipsec_config process that executes the keyadm commands is started from the ipipsec_configure( ) function in **ipipsec_config.c**.

It is surrounded by some ifdefs, as shown below:

```
.
.
 #if !defined(IP_PORT_LKM)&&  !defined(IP_PORT_OSE5)&&
 !defined(IPCOM_FORWARDER_NAE)
 #if !defined(IPWRNAD)&&  !defined(IP_PORT_VXWORKS)
      (void)ipcom_proc_create("ipipsec_config",
```

```
(Ipcom_proc_func)ipipsec_config, IPCOM_PROC_STACK_DEFAULT, IP_NULL);
#endif /* !IPWRNAD&&  !IP_PORT_VXWORKS
```

By removing

```
!defined(IP_PORT_VXWORKS)
```

it should start the process automatically when running VxWorks.

It is also possible to spawn a new task from command line using the 'sp' command, for example:

```
sp ipipsec_config
```

3. Editing the Program

About This Task

Prepare separate versions of the program for Target 1 and Target 2, as follows:

Procedure

1. For each target, configure a tunnel-mode connection, define IPIPSEC_USE_TUNNEL_EXAMPLE.

   Leaving this macro undefined would result in a transport mode connection.

2. For each target, configure the target with either AH or ESP SAs by defining IPIPSEC_USE_AH_EXAMPLE or IPIPSEC_USE_ESP_EXAMPLE respectively.

   To configure the target with both AH and ESP SAs, that is, to enable it to handle either type of traffic, you would define IPIPSEC_USE_ESP_AH_EXAMPLE.

3. Edit the keyadm command options in the section of the program relevant to the selected use case.
   1. For Target 1, set -src and -srcflow to the IP address of Target 1, and -dst and -dstflow to the address of Target 2.
   2. For Target 2, reverse these assignments and set -src and -srcflow to the IP address of Target 2, and -dst and -dstflow to the address of Target 1.

   You can specify IPv4 or IPv6 addresses.

4. Launching and Testing the Program

About This Task

Perform the following on each target:

Procedure

1. Download the VxWorks image and boot the target as described in Booting the Target on page 30.
2. Build, download, and launch the appropriate version of the configuration program.

   For instructions, see the *VxWorks 7 Configuration and Build Guide*

3. Test the SAs, flows, and groups that the program created.
   1. Log on to one of the targets.

      See Logging On to the Target on page 32 for more information.

      2. Issue keyadm shell commands to test, add, and delete SAs, flows and groups.

      For more information on keyadm, see keyadm Command Reference on page 35 and the *IPSec and IKE API reference*.

5. Booting the Target

The procedure for booting the target is the same for all example implementations.

About This Task

Switch on the target. The target downloads the VxWorks image. The image loads into memory and Wind River IPsec starts.

Procedure

1. Place the VxWorks image on an FTP server that is accessible from the target.
2. From a host, open a serial console session to COM1 of the target.
3. Switch on the target.

   The target downloads the VxWorks image. The image loads into memory and Wind River IPsec starts.

   The following appear on the console:

   - listing of boot parameters
   - attaching to network interface card
   - downloading VxWorks image from FTP server
   - initializing Wind River IPsec
   - theVxWorks banner

6. Changing Target Boot Parameters

You may need to change one or more of the target's boot parameters, such as the IP address of the FTP server.

About This Task

Procedure

1. Start the boot process.

   If the target is already running, reboot it by pressing **ctrl-x** on the console.

   See Booting the Target on page 30 for more information.

   The auto-boot message appears after a few seconds.

2. When the prompt "Press any key to stop auto-boot..." appears, press any key to stop the boot process.

   You have eight seconds to press a key, after which the boot process resumes.

3. At the "[VxWorks Boot]:" prompt, type **p** to list the boot parameters.

   Following are brief descriptions of some of the parameters:

   file name

   VxWorks image to be downloaded to target.

   inet on ethernet

   public IP address of target

   host inet

   IP address of FTP server

   user

   user name for accessing FTP server

   ftp password

   password for accessing FTP server

   startup script

   configuration script to be run after boot (if any)

4. Type **c** to start changing parameters.
5. Press **Enter** multiple times to list through the parameters.

   Pressing **Enter** leaves the parameter value unchanged.

6. To change a parameter value, type the new value and press **Enter**.

   When all changes are done and the "[VxWorks Boot]:" prompt appears, type **p** to check the new parameter values.

   The target saves the new values and uses them at the next boot.

7. Type **@** to resume the boot process.

Next Steps

It may be necessary to reboot the target between tests, so that the target can be in a known state. To reboot, press **`ctrl-x`** on the console. The auto-boot message appears after a few seconds.

7. Logging On to the Target

You can connect to the target using Telnet or transfer files using FTP. For more details about Telnet and FTP, see the *VxWorks 7 Network Stack Programmer's Guide*

Telnet connects to the VxWorks shell using port 23. The default username and password for the VxWorks shell are set at build time. Telnet connects to the IPCOM shell using port 2323.

By default, the username and password are not set. By default, the FTP server on the target has the username **anonymous** with no password. This is a read-only account.

Note: If the target runs IPsec but the host from which you connect to the target does not, you must disable output packet filtering on the target.

6. APPENDIX

1. ipsecctrl Command Reference

Several ipsecctrl shell commands list information from the IPsec task.

They can also be used to set global IPsec variables. You can run these commands at any time from the IPCOM shell on a target host.

Usage examples use the following target IP addresses:

- local target: 10.1.2.100 (IPv4), fec0:0:0:1::64 (IPv6)
- remote target: 10.1.2.56 (IPv4), fec0:0:0:1::1 (IPv6)

ipsecctrl flows

Description

This command is used to print all the IPsec output flows and their associated SAs.

Syntax

```
ipsecctrl flows
```

Example

Print all the IPsec output flows:

```
#0> ipsecctrl flows
Flows:
[1] :: use src=192.168.0.2/32:any dst=192.168.0.1/32:any any -> SA: ESP  spi=0x603 dst=19
2.168.0.1
[2] :: use src=192.168.0.1/32:any dst=192.168.0.2/32:any any <- SA: ESP  spi=0x603 dst=19
2.168.0.2
[3] :: use src=FEC0::1:0:0:0:64/128:any dst=FEC0::1:0:0:0:1/128:any any -> SA: ESP  spi=0
x703
dst=FEC0::1:0:0:0:1 [4] :: use src=FEC0::1:0:0:0:1/128:any dst=FEC0::1:0:0:0:64/128:any a
ny <-
SA: ESP  spi=0x703 dst=FEC0::1:0:0:0:64 Total of 4 flows.
```

ipsecctrl sa

Description

This command is used to print all the IPsec Security Associations together with some basic SA information.

Syntax

```
ipsecctrl sa
```

Example

Print the IPsec Security Associations:

```
#0> ipsecctrl sa
SA - Security Associations:
[ 1] ::  output ESP  spi=0x603   dst=192.168.0.1      auth=md5 hmac enc=espAesCbc
[ 2] ::  input  ESP  spi=0x603   dst=192.168.0.2      auth=md5 hmac enc=espAesCbc
```


WNDRVR

```
[ 3] ::  output AH   spi=0x701   dst=FEC0::1:0:0:0:1  auth=md5 hmac
[ 4] ::  input  AH   spi=0x701   dst=FEC0::1:0:0:0:64 auth=md5 hmac
[ 5] ::  output ESP  spi=0x703   dst=FEC0::1:0:0:0:1  auth=md5 hmac enc=espAesCbc
           ->   AH   spi=0x701   dst=FEC0::1:0:0:0:1
[ 6] ::  input  ESP  spi=0x703   dst=FEC0::1:0:0:0:64 auth=md5 hmac enc=espAesCbc
Total of 6 SAs.
```

ipsecctrl mib

Description

This command is used to print the IPsec monitor MIB from **draftietf-ipsec-monitor-mib-03.txt.**

Syntax

```
ipsecctrl mib
```

Example

Print the IPsec monitor MIB:

```
#0> ipsecctrl mib
```

ipsecctrl get

Description

This command is used to print IPsec configuration variables. Available configuration variables are:

- *esp_enable*
- *ah_enable*
- *ipip_allow*
- *enable_outfilter*

For explanations of the configuration variables, see Programming the Initialization Code on page 22.

Syntax

```
ipsecctrl get
```

Example

Print the IPsec config variables:

```
#0> ipsecctrl get
IPIPSEC configuration variables:
enable_outfilter = 0
esp_enable = 1
ah_enable = 1
ipip_allow = 1
```

ipsecctrl set

Description

This command is used to change IPsec configuration variables. Available configuration variables are:

- *esp_enable*

- *ah_enable*
- *ipip_allow*
- *enable_outfilter*

See Configuring a VxWorks Image Project on page 20 for explanations of the configuration variables.

Syntax

```
ipsecctrl set name value
```

Example

Disable input filter:

```
#0> ipsecctrl set esp_enable 1
ipsecctrl: esp_enable set to 1
```

2. keyadm Command Reference

The keyadm shell commands perform the several functions.

- add, delete, find, and modify SAs
- group SAs into SA bundles
- add, delete, find or dump output and input flows

For a demonstration of keyadm commands, see the following example configuration program:

installDir **/vxworks-7/pkgs_v2/net/ipnet/ipsecike-***x.x.x.x* **/ipsec/src/ipipsec_example.c**

This program uses hard-coded commands to configure IPsec at startup. If desired, you can modify the example code to read the commands from a file.

keyadm add

Description

This command adds a new SA to the IPsec SADB. The SA identifier consists, at a minimum, of the SPI, source and destination IP addresses, and security protocol. You must supply all desired modifiers, since the command creates the SA in the mature state, ready for input and output processing. Note that if -ingress or -egress is omitted, the SA direction defaults to output.

Mandatory Selectors

-spi *val*

val is the security parameter index (SPI), in hex without the initial 0x. For example:

```
-spi 500
```

specifies the value 0x500 hex (1280 decimal).

-src *ip*

Source IP address, in dot notation.

-dst *ip*

Destination IP address, in dot notation.

-proto *name*

Security protocol, either **esp**, **ah**, or **ipip**.

Optional Selectors

-sapri *val*

val is the (optional) priority (-128=highest to 127=lowest), used to add multiple SAs with the same identifiers (SPI, source and destination IP address, and security protocol) and to prioritize flows for output lookups.

-egress

Specifies an output SA (default).

-ingress

Specifies an input SA.

Mandatory Modifiers

-auth *alg*

alg is the authentication algorithm. Valid names are listed below.

- **md5** - HMAC-MD5
- **sha1** - HMAC-SHA1
- **rmd160** - HMAC-RIPEMD-160 message digests
- **aesmac** - HMAC-AES-XCBC-MAC
- **sha256** – HMAC-SHA-256
- **sha384** – HMAC- SHA-384
- **sha512** - HMAC-SHA-512

-akey *key*

key is the secret symmetric key used for authentication. Specify the key in hexadecimal digits; that is, four bits per hex digit. For example:

```
-akey 8317954752bf255b5321fe19e54a7f8d
```

specifies a 128-bit key (32 characters).

ESP only:

-enc *alg*

alg is the encryption algorithm. Valid names are listed below. All algorithms except none require the -ekey modifier to set the secret symmetric encryption key.

- **aes** - AES (Rijndael)
- **aesctr** - AES Counter
- **des** - DES
- **3des** - 3DES
- **blf** - Blowfish
- **cast** - CAST
- **none** - Null encryption; that is, no encryption.

-ekey *key*

The encryption key (in hex; each char is 4 bits).

key is the secret symmetric key used for encryption. For DES and 3DES, the key length is fixed at 8 and 24 octets respectively. For other ciphers, the key length is variable. Specify the key in hexadecimal digits; that is, four bits per hex digit. For example:

```
-ekey 8317954752bf255b5321fe19e54a7f8d
```

specifies a 128-bit key (32 characters).

Optional Modifiers

-proxy *ip*

A proxy IP address (for tunneling). Instruction to add a proxy IP address to packets.

Use of this modifier is recommended when host A communicates through a firewall to host B. For an incoming SA, the proxy address is the address of host B. To detect source spoofing attacks, host B checks the proxy address against the inner IP address of each packet. A proxy address is not necessary for outgoing SAs.

-replay *val*

val is the replay counter value supplied to the keyadm add and keyadm update commands. Allowed values are 0 to 255, where 0 (the default) disables the replay counter.

-forcetunnel

Forces IP-in-IP encapsulation before performing ESP or AH processing for an outgoing packet. The source and destination addresses of the packet are those supplied in the -src and -dst modifiers. If this flag is not supplied, IPsec still performs IP-in-IP encapsulation when necessary.

-ls *sec*

Soft “add” lifetime of the SA, in seconds. The “add” lifetime, as defined by RFC 2367, starts when the SA is added.

-lh *sec*

Hard “add” lifetime of the SA, in seconds. The “add” lifetime, as defined by RFC 2367, starts when the SA is added.

-bs *bytes*

Soft lifetime in bytes. The number of bytes that can be processed with this SA before it expires. At the end of the soft lifetime, a warning is sent that the SA is about to expire.

-bh *bytes*

Hard lifetime in bytes. The number of bytes that can be processed with this SA before it expires. At the end of the hard lifetime, the SA expires.

-ecn

Enables Explicit Congestion Notification (ECN).

-ens

Enables Extended Sequence Numbers (ESN) on the SA. This increases the sequence numbers from 32-bit to 64-bit.

-srcreq

Indicates that source IP must match in SA lookup.

-tfc

Instruction to use TFC to mask the traffic flow. This applies to ESP SAs only, not to AH SAs. You configure TFC using Workbench or by setting sysvars.

When you enable TFC using this command, a flow must exist between the source and destination of that particular SA in order for the dummy traffic to be allowed. If such a policy (or flow) is not created, the dummy packets are generated but dropped with an error message that no policy exists

-udpencap *port*

Enable ESP UDP encap using *port*. This applies to ESP SAs only.

-clrdf

Clear DF-bit on outer IPv4 header.

-setdf

Set DF-bit on outer IPv4 header.

-u

Specify the identifier of the policy entry in SPD (for use with keyadm spdadd only).

-linux

Enable Linux-style SA add. More specifically, this means adding a *struct sadb_x_sa2* structure with the SADB_ADD message, specifying tunnel or transport mode as well as optionally a policy identifier (-u).

Example

To add an output tunnel-mode IPv4 ESP SA using 3DES and MD5:

```
keyadm add -proto esp -enc 3des -auth md5 -spi 603
-dst 10.1.2.56 -src 10.1.2.100
-ekey b0e39876daf6d4ee5aaa895a4733bd8afdead83a2a8aa480
-akey 8317954752bf255b5321fe19e54a7f8d
-forcetunnel
-egress
```

To add an output transport-mode IPv6 AH SA using MD5:

```
keyadm add -proto ah -auth md5 -spi 701
-dst fec0:0:0:1::1 -src fec0:0:0:1::64
-akey 8317954752bf255b5321fe19e54a7f8d
-egress
```

keyadm delete

Description

This command deletes an existing SA and all associated flows. The SA identifier consists of the SPI, source and destination IP address, and security protocol. If the modifier -chain is supplied, the command deletes the entire SA chain.

Mandatory Selectors

-spi *val*

-spi is the security parameter index (SPI), in hex without the initial 0x. For example

```
-spi 500
```

specifies the value 0x500 hex (1280 decimal).

-src *ip*

Specifies the source IP address of the SA.

-dst *ip*

Specifies the destination IP address of the SA.

-proto *name*

name is the security protocol, either **esp**, **ah**, or **ipip**.

Optional Selectors

-sapri *val*

val is the (optional) priority (-128=highest to 127=lowest), used to identify a specific SA and flow based on its priority.

-egress

Specifies an output SA (default).

-ingress

Specifies an input SA.

Optional Modifiers

-chain

Instruction to the keyadm delete command to delete the entire SPI chain. If this flag is not supplied, the command deletes only the specified SPI.

Example

To delete a single input IPv4 ESP SA with SPI 603:

```
keyadm delete -spi 603 -src 10.1.2.110 -dst 10.1.2.100 -proto esp
```

To delete a single input IPv6 AH SA with SPI 701:

```
keyadm delete -spi 701 -src fec0:0:0:1::1 -dst fec0:0:0:1::64 -proto ah
```

keyadm dump

Description

This command lists all SAs. For security reasons, the SA keys do not appear in the list. The -proto modifier can be supplied to limit the list to the specified protocol.

Mandatory Modifiers

None

Optional Modifiers

-proto *name*

Use this to specify the security protocol, either **esp**, **ah**, or **ipip**, to limit the list to SAs matching this security protocol.

Example

To list all ESP SAs:

```
#0> keyadm dump -proto esp
SADB_MSG: msgtype=DUMP  err=0  satype=ESP  len=112  seq=27  pid=0x12
      SA: spi=503  sta=MATURE  auth=MD5  enc=AES  replay=32  ingress tfc esn
     SRC: addr=10.1.2.110
     DST: addr=10.1.2.100
 CURRENT: addtime=0 usetime=0 bytes=0
----------------------------------------
SADB_MSG: msgtype=DUMP  err=0  satype=ESP  len=112  seq=27  pid=0x12
      SA: spi=503  sta=MATURE  auth=MD5  enc=AES  replay=32  tunnel tfc esn setdf
     SRC: addr=10.1.2.100
     DST: addr=10.1.2.110
 CURRENT: addtime=0 usetime=0 bytes=0
----------------------------------------
SADB_MSG: msgtype=DUMP  err=0  satype=ESP  len=144  seq=27  pid=0x12
      SA: spi=703  sta=MATURE  auth=MD5  enc=AES  replay=0  ingress
     SRC: addr=fec0::1:0:0:0:1
     DST: addr=fec0::1:0:0:0:64
 CURRENT: addtime=0 usetime=0 bytes=0
----------------------------------------
SADB_MSG: msgtype=DUMP  err=0  satype=ESP  len=144  seq=0  pid=0x12
      SA: spi=703  sta=MATURE  auth=MD5  enc=AES  replay=0  tunnel
     SRC: addr=fec0::1:0:0:0:64
     DST: addr=fec0::1:0:0:0:1
 CURRENT: addtime=0 usetime=0 bytes=0
----------------------------------------
Total of 4 security associations
```

keyadm dumpflow

Description

This command lists all flows. For security reasons, the SA keys do not appear in the list. The -ingress or -egress options can be used to list only input or output flows, respectively.

Mandatory Modifiers

None

Optional Modifiers

-egress

Only list output flows.

-ingress

Only list input flows.

Example

To list all output flows:

```
#0> keyadm dumpflow -egress
SADB_MSG: msgtype=X_DUMPFLOW  err=0  satype=ESP  len=136  seq=30  pid=0x12
PROTOCOL: proto=any
 SRCFLOW: addr=10.1.2.100/32
 DSTFLOW: addr=10.1.2.90/32
```

```
 FLOWOPT: type=use prio=0 dscp=0x0 dir=out id=2
-----------------------------------------
SADB_MSG: msgtype=X_DUMPFLOW  err=0  satype=ESP  len=136  seq=30  pid=0x12
PROTOCOL: proto=any
 SRCFLOW: addr=10.1.2.100/32
 DSTFLOW: addr=10.1.2.110/32
 FLOWOPT: type=use prio=0 dscp=0x0 dir=out id=4
-----------------------------------------
SADB_MSG: msgtype=X_DUMPFLOW  err=0  satype=ESP  len=200  seq=0  pid=0x12
PROTOCOL: proto=any
 SRCFLOW: addr=fec0::1:0:0:0:64/128
 DSTFLOW: addr=fec0::1:0:0:0:1/128
 FLOWOPT: type=use prio=0 dscp=0x0 dir=out id=5
-----------------------------------------
Total of 3 flows
```

keyadm flush

Description

This command deletes all SAs and flows. The -proto modifier can be supplied to limit the deletion to the specified security protocol, for example AH or ESP. To delete only bypass flows, and not the SAs, specify the -proto bypass modifier.

Mandatory Modifiers

None

Optional Modifiers

-proto *name*

name is the security protocol, either **esp**, **ah**, **ipip**, or **bypass**.

Example

To delete all SAs and bypass flows:

```
#0> keyadm flush
SADB_MSG: typ=FLUSH  err=0  sat=NONE  len=16  seq=14  pid=0xe
-----------------------------------------

Output from syslog:
Tue Nov 25 14:02:30 2008: ipipsec[7]: Info: Removing SA: bypass #0>
Tue Nov 25 14:02:30 2008: ipipsec[7]: Info: Removing SA: ESP  out spi=0x603 dst=192.168.0
.1
Tue Nov 25 14:02:30 2008: ipipsec[7]: Info: Removing flow [output use 'prot=any src=192.1
68.0.2/32:0 dst=192.168.0.1/32:0' -> SA: ESP  out spi=0x603 dst=192.168.0.1] : ok
Tue Nov 25 14:02:30 2008: ipipsec[7]: Info: Removing SA: ESP  in spi=0x603 dst=192.168.0.
2
Tue Nov 25 14:02:30 2008: ipipsec[7]: Info: Removing flow [input use 'prot=any src=192.16
8.0.1/32:0 dst=192.168.0.2/32:0' -> SA: ESP  in spi=0x603 dst=192.168.0.2] : ok
Tue Nov 25 14:02:30 2008: ipipsec[7]: Info: Removing SA: AH   out spi=0x701 dst=FEC0::1:0
:0:0:1
Tue Nov 25 14:02:30 2008: ipipsec[7]: Info: Removing SA: ESP  out spi=0x703 dst=FEC0::1:0
:0:0:1
Tue Nov 25 14:02:30 2008: ipipsec[7]: Info: Removing flow [output use 'prot=any src=FEC0:
:1:0:0:0:64/128:0 dst=FEC0::1:0:0:0:1/128:0' -> SA: ESP  out spi=0x703 dst=FEC0::1:0:0:0:
1] : ok
Tue Nov 25 14:02:30 2008: ipipsec[7]: Info: Removing SA: AH   in spi=0x701 dst=FEC0::1:0:
```

```
0:0:64
Tue Nov 25 14:02:30 2008: ipipsec[7]: Info: Removing SA: ESP  in spi=0x703 dst=FEC0::1:0:
0:0:64
Tue Nov 25 14:02:30 2008: ipipsec[7]: Info: Removing flow [input use 'prot=any src=FEC0::
1:0:0:0:1/128:0 dst=FEC0::1:0:0:0:64/128:0' -> SA: ESP  in spi=0x703 dst=FEC0::1:0:0:0:64
] : ok
```

To delete all output bypass flows:

```
#0> keyadm flush -proto bypass
Tue Nov 25 14:01:47 2008: ipipsec[7]: Info: Removing SA: bypass
SADB_MSG: typ=FLUSH  err=0  sat=BYPASS  len=16  seq=13  pid=0xe
-----------------------------------------
```

keyadm get

Description

This command lists a specific SA. The SA identifier consists of the SPI, source and destination IP addresses, and security protocol.

Mandatory Selectors

-spi *val*

spi is the security parameter index (SPI), in hex without the initial 0x. For example:

```
-spi 500
```

specifies the value 0x500 hex (1280 decimal).

src *ip*

Specifies the source IP address of the SA.

dst *ip*

Specifies the destination IP address of the SA.

-protp *name*

name is the security protocol, either **esp**, **ah**, or **ipip**.

Optional Selectors

-sapri *val*

val is the (optional) priority (-128=highest to 127=lowest), used to prioritize flows for output lookups.

-egress

Specifies an output SA (default).

-ingress

Specifies an input SA.

Optional Modifiers

None

Example

To list an output IPv4 ESP SA:

```
#0> keyadm get -src 10.1.2.100 -spi 503 -dst 10.1.2.110 -proto esp -egress
```

If an SA with the specified SPI, source, destination, and security protocol does not exist, the command displays the following output:

```
#0> keyadm get -spi 601 -src 10.1.2.100 -dst 10.1.2.56 -proto ah
```

```
-------------------------------------------------------------------------------------
------------------------------
[vxWorks *]# keyadm get -spi 601 -src 10.1.2.100 -dst 10.1.2.56 -proto
                  ah SADB_MSG: msgtype=GET err=3 satype=AH len=16 seq=1
                    pid=0x88a50be0    ERRNO: S_errno_ESRCH (3)
-------------------------------------------------------------------------------------
--------------------------------
```

To list an output IPv6 ESP SA:

```
#0> keyadm get -spi 703 -src fec0:0:0:1::64 -dst fec0:0:0:1::1 -proto esp -egress
```

keyadm getspi

Description

This command creates a larval SA for a given security protocol, source address, and destination address. The IPsec task sets the SPI for the new SA. If successful, this command prints the resulting SA with its allocated SPI.

An SA created by keyadm getspi remains in the larval state until updated by the keyadm update command. The larval SA cannot be used for input or output IPsec processing. If the keyadm update command is not issued within a certain time (default 10 minutes) the larval SA is automatically deleted.

Creating an SA using keyadm getspi followed by keyadm update is an alternative to using keyadm add. The main advantage of using keyadm getspi is that the IPsec task, rather than the user, sets the SPI.

Mandatory Selectors

-scr *ip*

Specifies the source IP address.

-dst *ip*

Specifies the destination IP address.

-proto *name*

name is the security protocol, either **esp**, **ah**, or **ipip**.

Optional Selectors

-sapri *val*

val is the (optional) priority (-128=highest to 127=lowest), used to prioritize flows for output lookups.

-egress

Specifies an output SA (default).

-ingress

Specifies an input SA.

Mandatory Modifiers

-minspi *val*

val is the minimum security parameter index (SPI), in hex without the initial 0x. For example:

```
-minspi 500
```

specifies the value 0x500 hex.

-maxspi *val*

Specifies the maximum SPI value (in hex).

Optional Modifiers

None

Example

To create a larval input IPv4 AH with an SPI value from 1000 to 2000 chosen by the IPsec task:

```
#0> keyadm getspi -src 10.1.2.56 -dst 10.1.2.100 -proto ah -minspi 1000 -maxspi 2000 -ing
ress
SADB_MSG: msgtype=GETSPI  err=0  satype=AH  len=80  seq=21  pid=0xf
      SA: spi=1000  sta=LARVAL  auth=NONE  enc=NONE  replay=0
     SRC: addr=10.2.1.56
     DST: addr=10.1.2.100
----------------------------------------
```

keyadm monitor

Description

This command sends a PFKEY register message to the IPsec kernel and starts subscribing to PFKEY messages sent by the kernel. The messages are printed on the screen.

Mandatory Modifiers

None

Optional Modifiers

-c *count*

Maximum number of packets received.

-W *timeout*

Maximum timeout (in seconds) per read.

-proto *name*

name is the security protocol, either **esp**, **ah**, or **ipip**.

Example

To print the next three IPIPSEC kernel messages:

```
#0> keyadm monitor -c 3
SADB_MSG: msgtype=FLUSH  err=0  satype=NONE  len=16  seq=1604  pid=0x48a
----------------------------------------
SADB_MSG: msgtype=ADD  err=0  satype=ESP  len=112  seq=1605  pid=0x484
      SA: spi=2000  sta=MATURE  auth=MD5  enc=DES  replay=4294967296
     SRC: addr=2005:140::1:1
     DST: addr=2005:140::1:2
----------------------------------------
SADB_MSG: msgtype=X_ADDFLOW  err=0  satype=ESP  len=232  seq=1606  pid=0x484
      SA: spi=2000  sta=LARVAL  auth=NONE  enc=NONE  replay=4294967296
     DST: addr=2005:140::1:2
 SRCFLOW: addr=2005:140::1:1/128
 DSTFLOW: addr=2005:140::1:2/128
----------------------------------------
```

keyadm update

Description

This command updates an existing SA. The SA identifier consists of the SPI, source IP address, destination IP address, and security protocol.

The information that the command is allowed to change depends on the state of the SA. For larval SAs created by keyadm getspi, any SA information can be updated, whereas for mature SAs created by keyadm add, only the SA state and lifetimes can be changed.

If supplied with sufficient valid information, this command changes a larval SA to the mature state.

Note: The values used in an update must be valid or the SA will not be updated. You can use the keyadm get command to check for the existence of specific values.

Mandatory Selectors

-spi *val*

val is the security parameter index (SPI), in hex without the initial 0x. For example:

```
-spi 500
```

specifies the value 0x500 hex.

-src *ip*

Specifies the source IP address.

-dst *ip*

Specifies the destination IP address.

-proto *name*

name is the security protocol, either **esp**, **ah**, or **ipip**.

Optional Selectors

-sapr *val*

val is the (optional) priority (-128=highest to 127=lowest), used to prioritize flows for output lookups.

-egress

Specifies an output SA (default).

-ingress

Specifies an input SA.

Mandatory Modifiers

-auth *alg*

alg is the authentication algorithm. Valid names are listed below.

- **md5** - HMAC-MD5
- **sha1** - HMAC-SHA1
- **rmd160** - HMAC-RIPEMD-160 message digests
- **aesmac** - HMAC-AES-XCBC-MAC
- **sha256** – HMAC-SHA-256
- **sha384** – HMAC- SHA-384
- **sha512** - HMAC-SHA-512

-akey *key*

Sets the authentication key (in hex, each char is 4 bits).

-enc *alg*

This applies to ESP only. name is the encryption algorithm. Valid names are listed below. All algorithms except none require the -ekey modifier to set the secret symmetric encryption key.

- **aes** - AES (Rijndael)
- **des** - DES
- **3des** - 3DES
- **blf** - Blowfish
- **cast** - CAST
- **aesctr** - AES CTR
- **none** - Null encryption; that is, no encryption.

-ekey *key*

Sets the encryption key (in hex, each char is 4 bits). This applies to ESP only.

Optional Modifiers

-proxy *ip*

Specifies the Proxy IP address (for tunneling).

-replay *val*

val is the replay counter value supplied to the keyadm add and keyadm update commands. Allowed values are 0 to 255, where 0 (the default) disables the replay counter.

-forcetunnel

Forces tunneling mode.

Forces IP-in-IP encapsulation before performing ESP or AH processing for an outgoing packet. The source and destination addresses of the packet are those supplied in the -src and -dst modifiers. If this flag is not supplied, IPsec still performs IP-in-IP encapsulation when necessary.

-esn

Enable Extended Sequence Numbers (ESN) on the SA. This increases the sequence numbers from 32-bit to 64-bit.

-srcreq

Specifies that the source IP must match in SA lookup.

-tfc

Instruction to use TFC to mask the traffic flow. This applies to ESP SAs only, not to AH SAs. You configure TFC using Workbench or by setting sysvars.

When you enable TFC using this command, a flow must exist between the source and destination of that particular SA in order for the dummy traffic to be allowed. If such a policy (or flow) is not created, the dummy packets are generated but dropped with an error message that no policy exists.

-udpencap *port*

Enable ESP UDP encap using *port*. This applies to ESP SAs.

-clrdf

Clears the DF-bit on outer IPv4 header.

-setdf

Sets the DF-bit on outer IPv4 header.

-ls *sec*

Soft "add" lifetime of the SA, in seconds. The "add" lifetime, as defined by RFC 2367, starts when the SA is added.

-lh *sec*

Hard "add" lifetime of the SA, in seconds. The "add" lifetime, as defined by RFC 2367, starts when the SA is added.

-bs *bytes*

Soft lifetime in bytes. The number of bytes that can be processed with this SA before it expires. At the end of the soft lifetime, a warning is sent that the SA is about to expire.

-bh *bytes*

Hard lifetime in bytes. The number of bytes that can be processed with this SA before it expires. At the end of the hard lifetime, the SA expires.

-ecn

Enables Explicit Congestion Notification (ECN). This is for ESP tunnel mode only.

Example

To update an input IPv4 AH SA:

```
#0> keyadm update -spi 1000 -src 10.1.2.56 -dst 10.1.2.100 -proto ah \
-ingress -auth md5 -akey 8317954752bf255b5321fe19e54a7f8d
```

keyadm addflow

Description

This command creates a flow and optionally associates it with a specific SA. The SA identifier consists at a minimum of the SPI, source and destination addresses, and security protocol.

A flow is a set of packet selectors consisting of source and destination IP addresses, and additional optional selectors such as source and destination IP masks, source port, destination port, IP protocol identifier, etc.

There are five different flow types that can be specified. For descriptions of the various flow types, see Flow on page 12.

The -replace modifier causes the new flow to replace any existing duplicate flow (which may refer to a different SA).

Mandatory Selectors

-srcflow *ip*

ip is the source flow IP address. Combined with -srcmask to specify the source address or subnet. Packets must match this address to use the specified SA.

-dstflow *ip*

Specifies the destination flow address. Combined with -dstmask to specify the destination address or subnet.

Optional Selectors

-srcmask *mask*

mask is the source flow IP subnet mask (default mask is all ones).

-dstmask *mask*

mask is the destination flow mask (default is all ones).

-egress

Specifies that the flow is an output flow (default).

-ingress

Specifies that the flow is an input flow.

-srcmaxflow *ip*

Sets the source flow max address (for linear range).

-dstmaxflow *ip*

Sets the destination flow max address (for linear range).

-flowpri *prio*

prio is the (optional) priority (-128=highest to 127=lowest), used to prioritize flows for output lookups.

-dscp *val*

DSCP value (in hex without the 0x).

Optional IP Protocol Specific Selectors

-transport *val*

val is the IANA IP protocol number (ANY=0, TCP=6, UDP=17, and so forth). By default, the protocol number is not used for packet matching. If it is used, packets must match the protocol to use the specified SA.

-srcport *port*

port is the UDP/TCP source port number. By default, the source port is not used for packet matching. If it is used, packets must match this port to use the specified SA.

-dstport *port*

port is the UDP/TCP destination port number. By default, the destination port is not used for packet matching. If it is used, packets must match this port to use the specified SA.

-srcmaxport *port*

port is the maximum of a UDP/TCP source port range.

-dstmaxport *port*

port is the UDP/TCP destination port number. By default, the destination port is not used for packet matching. If it is used, packets must match this port to use the specified SA.

-icmptype *val*

val is the ICMP type number. By default, the ICMP type is not used for packet matching. If it is used, ICMP packets must match this type to use the specified SA.

-icmpcode *val*

val is the ICMP code number. By default, the ICMP code is not used for packet matching. If it is used, ICMP packets must match this code to use the specified SA.

-mhtype *val*

val is the Mobility Header (MH) type, specified as a value. By default, the MH type is not used for packet matching. If it is used, packets must match this type to use the specified SA.

Optional Type

-use

Specifies a use flow (default).

-bypass

Specifies a bypass flow.

-deny

Specifies a deny flow.

-acquire

Specifies an acquire flow.

-require

Specifies a require flow.

Optional SA Selectors

-spi *val*

val is the security parameter index (SPI), in hex without the initial 0x. For example:

```
-spi 500
```

specifies the value 0x500 hex.

-scr *ip*

Specifies the source IP address in dot notation.

-dst *ip*

Specifies the destination IP address.

-proto *name*

name is the security protocol, either **esp**, **ah**, or **ipip**.

-sapri *val*

val is the (optional) priority (-128=highest to 127=lowest), used to prioritize flows for output lookups.

Optional Modifier

-replace

Instruction to replace the flow if it is a duplicate.

Example

```
keyadm addflow -spi 503 -src 10.1.2.100 -dst 10.1.2.110 -proto esp -srcmask 255.255.255.2
55 -dstmask 255.255.255.255 -srcflow 10.1.2.100 -dstflow 10.1.2.100
SADB_MSG: msgtype=X_ADDFLOW  err=0  satype=ESP  len=176  seq=17  pid=0xf
      SA: spi=503  sta=LARVAL  auth=NONE  enc=NONE  replay=0
     SRC: addr=10.1.2.100
     DST: addr=10.1.2.110
 SRCFLOW: addr=10.1.2.100/32
 DSTFLOW: addr=10.1.2.100/32
----------------------------------------
```

To replace an IPv6 input flow (for all traffic between two IPv6 targets) associated with an input IPv6 AH SA:

```
keyadm addflow -spi 703 -src fec0:0:0:1::1 -dst fec0:0:0:1::64 -proto esp -srcmask ffff:f
fff:ffff:ffff:ffff:ffff:ffff:ffff -dstmask ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffff -srcfl
ow fec0:0:0:1::1 -dstflow fec0:0:0:1::64 -ingress -replace
SADB_MSG: msgtype=X_ADDFLOW  err=0  satype=ESP  len=232  seq=22  pid=0xf
      SA: spi=703  sta=LARVAL  auth=NONE  enc=NONE  replay=0  replaceflow ingress
     DST: addr=fec0::1:0:0:0:64
 SRCFLOW: addr=fec0::1:0:0:0:1/128
 DSTFLOW: addr=fec0::1:0:0:0:64/128
----------------------------------------
```

The following examples show how to add bypass flows.

This is an example of a bypass flow to bypass traffic on UDP port 17185 (host IP address =192.168.200.254).

```
[vxworks]# keyadm addflow -bypass -srcmask 255.255.255.255 -dstmask 255.255.255.255 -srcf
low 192.168.200.254 -dstflow 192.168.200.14 -transport 17 -dstport 17185
```

This is an example of a bypass flow that bypasses the Performance Profiler Traffic on TCP port 5678.

```
[vxworks]# keyadm addflow -bypass -srcmask 255.255.255.255 -dstmask 255.255.255.255 -srcf
low 192.168.200.254 -dstflow 192.168.200.14 -transport 6 -dstport 5678
```

keyadm delflow

Description

This command deletes a flow. Note that if the flow points to an SA, the SA is not deleted. It must be specifically deleted using the keyadm delete command.

A flow is a set of packet selectors consisting of a source and destination IP addresses, source and destination IP masks, and other optional selectors, such as source port, a destination port, IP protocol identifier, etc.

There are five different flow types that can be specified. For descriptions of the various flow types, see Flow on page 12.

Mandatory Selectors

-srcflow *ip*

ip is the source flow IP address. Combined with -srcmask to specify the source address or subnet. Packets must match this address to use the specified SA.

-srcmask *mask*

mask is the source flow IP subnet mask. Must be supplied with -srcflow.

-dstflow *ip*

ip is the destination flow IP address.

-dstmask *mask*

mask is the destination flow mask.

Optional Selectors

-egress

Specifies that the flow is an output flow (default).

-ingress

Specifies that the flow is an input flow.

-srcmaxflow *ip*

The source flow maximum address (for linear range).

-dstmaxflow *ip*

The destination flow maximum address (for linear range).

-flowpri *prio*

prio is the (optional) priority (-128=highest to 127=lowest), used to prioritize flows for output lookups.

-dscp *val*

DSCP value (in hex without the 0x).

WNDRVR

Optional IP Protocol Specific Selectors

-transport *val*

val is the IANA IP protocol number (ANY=0, TCP=6, UDP=17, etc.). By default, the protocol number is not used for packet matching. If it is used, packets must match the protocol to use the specified SA.

-srcport *port*

port is the UDP/TCP source port number. By default, the source port is not used for packet matching. If it is used, packets must match this port to use the specified SA.

-dstport *port*

port is the destination UDP/TCP port number. By default, the destination port is not used for packet matching. If it is used, packets must match this port to use the specified SA.

-srcmaxport *port*

port is the maximum of a UDP/TCP source port range.

-dstmaxport *port*

port is the maximum of a UDP/TCP destination port (for linear range).

-icmptype *val*

val is the ICMP type number. By default, the ICMP type is not used for packet matching. If it is used, ICMP packets must match this type to use the specified SA.

-icmpcode *val*

val is the ICMP code number. By default, the ICMP code is not used for packet matching. If it is used, ICMP packets must match this code to use the specified SA.

-mhtype *val*

val is the Mobility Header (MH) type, specified as a value. By default, the MH type is not used for packet matching. If it is used, packets must match this type to use the specified SA.

Example

To delete an IPv4 flow (for all traffic between two IPv4 targets):

```
keyadm delflow -srcmask 255.255.255.255 -dstmask 255.255.255.255 -srcflow 10.1.2.100 -dst
flow 10.1.2.110
SADB_MSG: msgtype=X_DELFLOW  err=0  satype=NONE  len=112  seq=32  pid=0xf
 SRCFLOW: addr=10.1.2.100/32
 DSTFLOW: addr=10.1.2.110/32
-----------------------------------------
```

To delete a IPv6 input flow (for all traffic between two IPv6 targets) associated with an input IPv6 AH SA:

```
keyadm delflow -srcmask ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffff -dstmask ffff:ffff:ffff:f
fff:ffff:ffff:ffff:ffff -srcflow fec0:0:0:1::1 -dstflow fec0:0:0:1::64 -ingress -replace
SADB_MSG: msgtype=X_DELFLOW  err=0  satype=NONE  len=192  seq=37  pid=0xf
      SA: spi=0  sta=LARVAL  auth=NONE  enc=NONE  replay=0  replaceflow ingress
 SRCFLOW: addr=fec0::1:0:0:0:1/128
 DSTFLOW: addr=fec0::1:0:0:0:64/128
-----------------------------------------
```

keyadm findflow

Description

This command looks up a flow and is mainly used for debugging purposes, since it can be used to check which flow is used for a specific IP datagram, specified using the selectors.

A flow is a set of packet selectors consisting of a source and destination IP addresses, source and destination IP masks, and other optional selectors, such as source port, a destination port, IP protocol identifier, etc.

There are five different flow types that can be specified. For descriptions of the various flow types, see Flow on page 12.

Mandatory Selectors

-srcflow *ip*

ip is the source flow IP address

-dstflow *ip*

ip is the destination flow IP address

-transport *val*

val is the IANA IP protocol number (ANY=0, TCP=6, UDP=17, etc.). By default, the protocol number is not used for packet matching. If it is used, packets must match the protocol to use the specified SA.

Optional Selectors

-egress

Specifies that the flow is an output packet (default).

-ingress

Specifies that the flow is an input packet.

-dscp *val*

Indicates a flow with a specific DSCP value (in hex without the 0x).

-srcport *port*

port specifies the source UDP/TCP port of the flow.

-dstport *port*

port specifies the destination UDP/TCP port number of the flow.

-icmptype *val*

val is the ICMP type number of the flow.

-icmpcode *val*

val is the ICMP code number of the flow.

-mhtype *val*

val is the Mobility Header (MH) type, specified as a value. By default, the MH type is not used for packet matching. If it is used, packets must match this type to use the specified SA.

Example

To check which flow is used for the specified IPv4 TCP output packet:

```
keyadm findflow -srcflow 10.1.2.100 -dstflow 10.1.2.110 -transport 6 -egress
SADB_MSG: msgtype=X_FINDFLOW  err=0  satype=ESP  len=136  seq=24  pid=0xf
PROTOCOL: proto=any
 SRCFLOW: addr=10.1.2.100/32
 DSTFLOW: addr=10.1.2.110/32
 FLOWOPT: type=use prio=0 dscp=0x0 dir=out id=10
-----------------------------------------
```

keyadm group

Description

This command groups two SAs together to form an SA bundle. An SA bundle may be necessary to implement a security policy that requires both AH and ESP, or that requires iterated or nested tunneling. Note that the group modifier can only be used with egress SAs.

The SAs in a bundle are applied to a packet in the order in which they were grouped. The leading SA, which is applied first, is specified by -spi, -src, -proto and -dst modifiers. The second SA is specified by -spi2, -proto2 and -dst2 modifiers. Further SAs can be added to the bundle by repeated use of the command.

Grouping an SA with itself results in the SA being ungrouped.

1st SA Mandatory Selectors

-spi *val*

val is the security parameter index (SPI), in hex without the initial 0x. For example, -spi 500 specifies the value 0x500 hex.

-scr *ip*

Specifies the source IP address in dot notation.

-dst *ip*

Specifies the destination IP address in dot notation.

-proto *name*

Specifies the security protocol, either **esp**, **ah**, or **ipip**.

1st SA Optional Selectors

-sapri *val*

val is the (optional) priority (-128=highest to 127=lowest), used to add multiple SAs with the same identifiers (SPI, source and destination IP address, and security protocol) and to prioritize flows for output lookups.

-egress

Specifies that this is an output SA (default).

-ingress

Specifies that this is an input SA.

2nd SA Mandatory Selectors

Note: The second SA is always outbound.

-spi2 *val*

spi2 is the second SPI supplied to the keyadm group command to specify the SPI of the second SA

-dst2 *ip*

Specifies the destination IP address of the second SA.

-proto2 *name*

Specifies the security protocol for the second SA, either esp, ah, or ipip.

Optional Modifiers

None

Example

To group two outgoing IPv4 SAs:

```
keyadm group -spi 603 -src 10.1.2.100 -dst 10.1.2.56 -proto esp -proto2 ah -spi2 601 -dst
2 10.1.2.56
```

Outgoing packets are first processed by the ESP SA, and then by the AH SA.

keyadm spdadd & spdupdate

Description

The keyadm spdadd command creates a flow using the Linux style PFKEY message SADB_X_SPDADD (instead of the BSD style message SADB_X_ADDFLOW used in keyadm addflow command).

The keyadm spdupdate command updates an existing flow using the SADB_X_SPDUPDATE message. Updating a flow is the same as removing an old matching flow and adding a new one, and is similar to using keyadm addflow with the -replace option.

A flow is a set of packet selectors consisting of source and destination IP addresses, and additional optional selectors such as source and destination IP masks, source port, destination port, IP protocol identifier, and so forth.

Note that Linux output use flows does not require an output SA. The best matching SA is looked up when a packet matches the output flow. If the policy entry identifier (set with -u) is used, only the SA with the same matching id is used (Linux setkey unique mode). There are five different flow types that can be specified. For descriptions of the various flow types, see Flow on page 12.

Mandatory Selectors

-srcflow *ip*

ip is the source flow IP address. Combined with -srcmask to specify the source address or subnet. Packets must match this address to use the specified SA.

-dstflow *ip*

Specifies the destination flow address. Combined with -dstmask to specify the destination address or subnet.

Optional Selectors

-srcmask *mask*

mask is the source flow IP subnet mask (default mask is all 1s).

-dstmask *mask*

mask is the destination flow mask (default all 1s).

-srctunnel *ip*

ip is the tunnel source address.

-dstunnel *ip*

ip is the tunnel destination address.

-egress

Specifies that the flow is an output flow (default). You need to specify either -egress or -ingress.

-ingress

Specifies that the flow is an input flow. You need to specify either -egress or -ingress.

-flowpr *prio*

prio is the (optional) priority (-128=highest to 127=lowest), used to prioritize flows for output lookups.

-u *id*

id is policy entry identifier used to link a flow to SA (setkey unique mode).

-proto *type*

Specifies the security protocol for the SA: **esp**, **ah**, or **ipip**.

Optional IP Protocol Specific Selectors

-transpor *val*

val is the IANA IP protocol number (ANY=0, TCP=6, UDP=17, and so forth). By default, the protocol number is not used for packet matching. If it is used, packets must match the protocol to use the specified SA.

-srcport *port*

port is the UDP/TCP source port number. By default, the source port is not used for packet matching. If it is used, packets must match this port to use the specified SA

-dstport *port*

port is the UDP/TCP destination port number. By default, the destination port is not used for packet matching. If it is used, packets must match this port to use the specified SA.

Optional Type

-use

Specifies a use flow (default).

-bypass

Specifies a bypass flow.

-deny

Specifies a deny flow.

acquire

Specifies an acquire flow.

-require

Specifies a require flow.

Example

This is an example of how to create an output flow pointing to the unique SA with policy identifier 123.

```
keyadm spdadd -spi 503 -src 10.1.2.100 -dst 10.1.2.110 -proto esp -srcmask 255.255.255.25
5 -dstmask 255.255.255.255 -srcflow 10.1.2.100 -dstflow 10.1.2.100 -egress -u 123
SADB_MSG: msgtype=LX_SPDADD  err=0  satype=ESP  len=96  seq=2  pid=0xe
     SRC: addr=10.1.2.100
     DST: addr=10.1.2.100
  POLICY: type=ipsec prio=0 dir=outbound id=1 len=16 proto=50 mode=1 level=3 reqid=123
```

keyadm spddelete

Description

This command deletes a Linux style flow.

A flow is a set of packet selectors consisting of a source and destination IP addresses, source and destination IP masks, and other optional selectors, such as source port, a destination port, IP protocol identifier, etc. There are five different flow types that can be specified.

For descriptions of the various flow types, see Flow on page 12.

Mandatory Selectors

-srcflow *ip*

ip is the source flow IP address. Combined with -srcmask to specify the source address or subnet. Packets must match this address to use the specified SA.

-dstflow *ip*

ip is the destination flow IP address.

Optional Selectors

-srcmask *mask*

mask is the source flow IP subnet mask. Must be supplied with -srcflow.

-dstmask *mask*

mask is the destination flow mask.

-srctunnel *ip*

ip is the tunnel source address.

-dstunnel *ip*

ip is the tunnel source address.

-egress

Specifies that the flow is an output flow (default).

-ingress

Specifies that the flow is an input flow.

-flowpri *prio*

prio is the (optional) priority (-128=highest to 127=lowest), used to prioritize flows for output lookups.

Optional IP Protocol Specific Selectors

-transport *val*

val is the IANA IP protocol number (ANY=0, TCP=6, UDP=17, etc.). By default, the protocol number is not used for packet matching. If it is used, packets must match the protocol to use the specified SA.

-srcport *port*

port is the UDP/TCP source port number. By default, the source port is not used for packet matching. If it is used, packets must match this port to use the specified SA.

-dstport *port*

port is the destination UDP/TCP port number. By default, the destination port is not used for packet matching. If it is used, packets must match this port to use the specified SA.

Example

This example deletes the flow added in the spdadd example:

```
keyadm spddelete -spi 503 -src 10.1.2.100 -dst 10.1.2.110 -proto esp -srcmask 255.255.255
.255 -dstmask 255.255.255.255 -srcflow 10.1.2.100  -dstflow 10.1.2.100 -egress
SADB_MSG: msgtype=LX_SPDDELETE  err=0  satype=ESP  len=96  seq=11  pid=0xe
     SRC: addr=10.1.2.100
     DST: addr=10.1.2.100
  POLICY: type=ipsec prio=0 dir=outbound id=5 len=16 proto=50 mode=1 level=1 reqid=0
```

keyadm spdflush

Description

This command deletes all flows using the Linux style SADB_X_SPDFLUSH PFKEY message. It deletes all input and output flows of all types. No SAs are deleted. Use keyadm flush to delete SAs.

Mandatory Modifiers

None

Optional Modifiers

None

Example

To delete all flows:

```
keyadm spdflush
SADB_MSG: msgtype=LX_SPDFLUSH  err=0  satype=NONE  len=16  seq=14  pid=0xe
-----------------------------------------
Output from syslog:
[ipipsec]: Debug: ipipsec_pfkeyv2_input() :: receiving 'LX_SPDFLUSH' command
[ipipsec]: Info: Removing flow [#6 output use 'prot=any src=10.1.2.100/32:0 dst=10.1.2.10
0/32:0' -> '-P out ipsec                                esp/transport//unique:123']
[ipipsec]: Debug: ipipsec_pfkeyv2_input() :: succeeded
```

7. GLOSSARY

3DES

Triple-DES

AES

Advanced Encryption Standard

AH

The Authentication Header protocol provides connectionless integrity, data origin authentication, and an optional anti-replay service. Unlike ESP, AH secures parts of the IP header such as the source and destination addresses.

authentication

In this guide, authentication refers to data origin authentication and connectionless integrity for IP packets.

CBC

Cipher Block Chaining

CCI

Common Cryptographic Interface

CTR

Counter encryption mode

default policy

The policy applied to packets for which there is no matching policy in the Security Policy Database (SPD), usually a bypass or discard policy.

DES

Data Encryption Standard

egress

Specifies the outgoing data side of a network peer.

encapsulation

The process of wrapping data in a protocol header. Specifically, the header and payload of one type of packet become the payload of another type of packet. For example, network traffic is wrapped in an Ethernet header before being transmitted over the network. When bridging dissimilar networks, the entire frame from one network is placed in the header used by the link layer protocol of the other network. See also tunnel mode.

ESP

The Encapsulating Security Payload protocol can provide confidentiality by encryption, and limited traffic flow confidentiality. It can also provide connectionless integrity, data origin authentication, and an anti-replay service.

flow

A flow is a 5-tuple set of packet selectors consisting of a source IP address, a destination IP address, a source port, a destination port, and an IP protocol identifier. In Wind River IPsec, flows describe traffic for the purpose of determining the security processing to be applied to the traffic.

HMAC

Keyed hashing for message authentication code

IKE

Internet Key Exchange is a protocol for exchanging keying material between two peers over an insecure network link. IKE is based on ISAKMP, Oakley and SKEME. There are two versions of IKE: version 1 (IKE v1), and version 2 (IKE v2).

Ingress

Specifies the incoming data side of a network peer.

IPCOM

Specifies the incoming data side of a network peer.

IPIP

IP-in-IP tunneling, used to encapsulate an IP packet in an outer IP header with protocol number 4. The inner packet is unpacked and parsed by the target specified by the outer destination IP address.

IPsec

Internet Protocol security

IV

Initialization vector or initializing value

lifetime

The configurable length of time after which IKE automatically deletes a protection suite, so that its repeated use does not pose a security risk.

MAC

Message authentication code

PF_KEY v2

PF_KEY v2 is a generic key management API definition. Its purpose is to provide an OS-independent interface between key management applications, such as Wind River IKE, and OS-specific key engines, such as the security association database (SADB) of Wind River IPsec.

protection suite

A list of the security services that must be applied by a security protocol. All the services in the list must be applied. For example, the protection suite for the ESP protocol may consist of 3DES encryption, and the protection suite for the AH protocol may consist of keyed MD5 (HMAC).

policy

A rule for applying security processing to packets, for example, a rule that applies a specific type of encryption to packets of a specific traffic type that are destined for a specific port on a host at a specific address.

proposal

A proposal is a set of one or more transforms. All transforms in a proposal must be of the same type: AH or ESP.

RTOS

Real-time Operating System.

SA

A security association is a contract between two network peers, negotiated by IKE or set manually, that governs how the peers protect data passing between them. An SA applies to packets of a specific traffic type passing between specific ports on hosts at specific addresses. It determines the security protocol (AH or ESP), protocol mode, and protocol options to be used by IPsec for those packets. It is uniquely identified by a 4-tuple consisting of a Security Parameter Index (SPI), the source address, the destination address, and the protocol. An AH SA specifies the authentication key and the hash algorithm to be used. An ESP SA specifies the encryption key and the encryption algorithm to be used.

SA bundle

A security association bundle is a sequence of SAs by which traffic must be processed to satisfy a security policy.

SADB

Security Association Database

selectors

IP and transport layer header information.

SHA-1

Secure Hashing Algorithm, version 1

SPD

Security Policy Database

SPI

A security parameter index is a 32-bit value included in ESP and AH headers, which is used to identify which SA should be applied to a packet.

tag

A 32-bit constant used to identify a configuration variable. A tag is always followed by a 32-bit variable that stores its value.

Tag array

A variable-length array of tags and their associated tag values. The array must be terminated with a tag having the value IPTAG_END.

tag value

The 32-bit storage that immediately follows a tag. The tag value stores the input or output value of the tag. The tag value is of type **Iptag** but may contain any 32-bit type, such as a pointer to a string.

target

A CPU on which VxWorks runs.

transport mode

An end-to-end IPsec mode that protects upper-layer protocols. The end-point computers do the security processing.

tunnel mode

A portal-to-portal IPsec mode that protects entire IP packets. A single node provides security to several computers. Tunnel mode employs the technique of encapsulation, which enables a packet from one network to be transported by another network, by adding an outer IP header that is accepted by the second network.